A Practical Guide to Building Better Websites

SEO, GEO, accessible design, user centered principles, usability heuristics, and launch review

Compiled from Ohad's degree coursework | 22 October 2026

Purpose: a single working reference for planning, building, and reviewing a public website. The guidance below consolidates the course materials listed in the source appendix; it is a synthesis, not a verbatim reproduction.

Build for three audiences at once: people, search engines, and systems that summarize web content. Clear structure and trustworthy content help all three.

1. Start with the user and the task

A good website begins with the people who will use it and the decisions they need to make. Before choosing colors or writing code, define the site's purpose, primary audiences, and top tasks.

- Write a one sentence purpose statement: who the site serves, what they can do, and why it matters.
- List the main user groups, including first time visitors, returning visitors, mobile users, and people using assistive technology.
- Choose the three to five tasks that matter most. Make each task easy to start from the home page and navigation.
- Map the user's likely mental model: what words, categories, and steps would they expect? Use familiar terms instead of internal jargon.
- Sketch the journey from entry page to completion. Identify confusing decisions, avoidable steps, and likely errors.

A practical page brief

Question	Decision to record
Who is this page for?	Primary audience, context, and accessibility needs.
What should they accomplish?	One main action, plus any secondary actions.
What information do they need?	Answer first, then supporting detail and evidence.
What could go wrong?	Missing data, unclear labels, failed forms, slow loading, small screens.

2. Information architecture and content

Organize content by the visitor's questions, not by the organization's internal structure. Each page needs a clear topic and a reason to exist.

- Give pages descriptive names and stable, readable URLs. Use navigation labels that predict the destination.

- Place the answer or key action near the top. Use short paragraphs, descriptive subheadings, and meaningful lists.
- Use one clear main heading for the page and a logical hierarchy of section headings. Do not choose heading levels merely for visual size.
- Connect related pages with descriptive links. A link should make sense even when read out of context.
- Show author or organization identity, sources, and an update date when trust and currency matter.
- Add a concise About page and useful FAQ where visitors genuinely have repeated questions.

For a large site, group related pages around a central topic page and link the supporting pages to it. This helps visitors explore and helps crawlers understand relationships.

3. Semantic HTML and a durable page structure

Use HTML elements for their meaning. Semantic structure helps screen readers, browsers, search engines, and future maintainers understand a page.

- Use **header**, **nav**, **main**, **section**, **article**, **aside**, and **footer** when their meanings fit.
- Use **button** for actions and **a** for navigation. Use lists for groups of related items and tables for tabular data.
- Set the document language. For Hebrew pages, also set the direction to right to left.
- Keep important explanatory content available in the page HTML where possible, especially in a single page application.
- Provide unique page titles and useful descriptions. Use canonical URLs when duplicate URLs can represent the same content.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Clear page title | Site name</title>
  <meta name="description" content="A concise description of this page.">
</head>
<body>
  <header>...</header>
  <nav aria-label="Main navigation">...</nav>
  <main><h1>Page topic</h1>...</main>
  <footer>...</footer>
</body>
</html>
```

4. Visual design and responsive layout

Visual design should support comprehension and task completion. Make the hierarchy obvious: what is most important, what belongs together, and what can be acted on.

- Use a small, consistent set of type sizes, colors, spacing values, and component styles.
- Keep text readable: comfortable line length, sufficient line spacing, clear typography, and strong foreground/background contrast.
- Use whitespace to group related items and separate different sections. Avoid visual clutter and competing calls to action.
- Make interactive elements look interactive. Give buttons and links consistent states for hover, focus, disabled, and selected conditions.

- Design for narrow screens from the start. Use fluid sizing, Flexbox or Grid, and breakpoints based on content rather than a single device.
- Check for horizontal overflow, clipped text, oversized media, and controls that become difficult to tap on mobile.

Responsive layout is more than fitting the screen: the same task should remain understandable and usable when the layout changes.

5. Accessibility essentials

Accessibility means that people with differences in vision, hearing, motor ability, or cognition can perceive, operate, and understand the site. The course materials emphasize integrating it during development rather than adding it at the end.

Structure and navigation

- Provide logical headings and landmarks. Include a way to skip repetitive navigation when appropriate.
- Make every interactive control reachable and usable from a keyboard. Keep focus order logical and focus visibly apparent.
- Use real links and buttons rather than clickable generic elements. Give icon only controls an accessible name.
- Do not communicate meaning with color alone; add text, icons, or patterns.

Text, images, media, and forms

- Aim for at least 4.5:1 contrast for ordinary text, as cited in the course notes; test actual colors rather than guessing.
- Write alternative text that conveys the function or meaning of informative images. Decorative images should be ignored by assistive technology.
- Provide captions for spoken video and transcripts when they help people access audio content.
- Associate every form input with a clear label. Explain required fields, formats, and errors in plain language.
- Announce important dynamic status changes accessibly without unexpectedly moving keyboard focus.

Fast manual checks

Test	What success looks like
Keyboard only	Tab reaches every control; Enter/Space activate it; focus stays visible.
Zoom and narrow viewport	Text and controls remain readable; nothing essential is clipped.
Screen reader sample	Headings, links, labels, and status messages make sense when announced.
Color check	Text contrast is sufficient and error/success states have noncolor cues.

6. Usability heuristics for everyday users

The user centered design notes combine ideas associated with Shneiderman's golden rules and Nielsen style heuristics. Treat these as a review lens, not a substitute for watching real users complete tasks.

Principle	Apply it on a website
Consistency	Keep labels, placement, patterns, and behavior predictable across pages.
Informative feedback	Show when an action was received, saved, completed, or failed.
Prevent errors	Constrain invalid input and explain format before submission.
User control	Allow cancel, back, edit, undo, and safe exits where appropriate.
Familiar language	Use words and concepts visitors already know.
Recognition over recall	Show choices and context instead of requiring memory.
Efficiency	Support shortcuts for experienced users without confusing beginners.
Minimalism	Show what is needed now; remove distracting or redundant content.

Mental models and interruptions

A visitor has an expectation about what will happen after each action. Match familiar conventions or make differences explicit. Proactive prompts can help, but place them at natural pauses rather than interrupting a focused task. Avoid surprising movement of controls, content, or navigation.

7. SEO: make useful pages discoverable

SEO is the practice of making relevant pages easy for search engines to discover, understand, and present. Its foundation is useful content, clear structure, and a technically accessible site.

On page fundamentals

- Write a unique, descriptive title for each page and a concise meta description that accurately previews it.
- Choose one primary topic per page. Use the topic naturally in the main heading, opening explanation, and relevant subheadings.
- Avoid keyword stuffing. Cover the real questions people have, with original detail and evidence.
- Use descriptive internal links and trustworthy references when they genuinely help the reader.
- Use meaningful file names and alternative text for informative images. Compress heavy media to protect loading speed.

Site level fundamentals

- Keep important pages reachable through navigation and links. Provide a sitemap when useful and manage crawler access deliberately.
- Use canonical URLs for duplicate content and avoid broken or unstable links.
- Check mobile layout, performance, and the actual published pages, not just local previews.
- Do not spend effort on meta keywords; the course material notes that Google does not use them for ranking.

8. GEO: make content easy to understand and cite

GEO refers to shaping content so AI based answer systems can interpret and potentially cite it. Citation behavior varies by system, so there is no guaranteed formula. The course material's strongest, most durable advice is to make facts explicit, well structured, and attributable.

- Answer the page's central question plainly near the top, then provide supporting explanation and examples.
- Use descriptive headings, clear definitions, factual statements, and concise question and answer sections where useful.
- Name the author or organization, identify sources, and show when time sensitive information was updated.
- Keep important facts in accessible text; supplement video or audio with captions or transcripts.
- Use structured data only when it accurately describes visible content. Keep names, dates, and URLs consistent.
- Treat optional experimental files such as llms.txt as secondary. They do not replace crawlable, high quality pages.

For a single page application, critical explanations should not depend entirely on JavaScript generated views. Give each section an understandable heading and keep navigation and status changes accessible.

9. Forms and interactive experiences

- Ask only for information needed to complete the task. Group related fields and label each one.
- Validate input at the right time; explain how to fix a problem without discarding entered data.
- Confirm important actions, especially saves, submissions, payments, or destructive changes.
- Make selected states visible in text or accessible state attributes, not only by color.
- Support correction and reversal where practical. Test empty, invalid, partial, and successful flows.
- For dynamic interfaces, announce important changes and manage focus intentionally.

10. Evaluate the site with evidence

The interface evaluation notes describe expert review, heuristic checks, consistency checks, and usability testing. Each catches different problems. A quick expert pass is useful, but it should be followed by observing representative users where possible.

A lightweight review sequence

Step	Action
1. Walk through	Complete the top tasks yourself on desktop and mobile; record friction.
2. Heuristic review	Check the eight principles in section 6 and note specific failures.
3. Accessibility pass	Test keyboard, focus, labels, contrast, headings, and media alternatives.
4. User observation	Ask representative people to complete realistic tasks without coaching.
5. Prioritize fixes	Rank issues by task impact, frequency, and severity; retest after changes.

For every issue, record where it occurs, the task being attempted, what happened, why it matters, and a possible repair. Distinguish an observed problem from an assumed one.

11. Prelaunch checklist

Area	Check
Purpose and content	Every important page has a clear purpose, answer, owner, and current information.
Navigation	Top tasks are easy to find; links describe destinations; no dead ends.
Visual design	Typography, spacing, colors, controls, and mobile layouts are consistent.
Accessibility	Keyboard use, focus, headings, labels, alt text, contrast, and captions are checked.
SEO	Titles, descriptions, URLs, links, crawlability, and published pages are checked.
GEO	Core facts are explicit, sourced, dated where needed, and readable as text.
Interaction	Empty, error, success, cancel, and recovery states work as intended.
Quality	HTML/CSS are validated; no broken media, clipped text, or console errors remain.

12. Source materials

This guide was synthesized from the following files in Ohad's degree folder. The course notes may include assignment specific rules; apply those only when doing that assignment.

The full PDF course summary, SEO reference, HTML5 and CSS3 reference sheets, and final project brief follow this guide as appendices.

Website development course summary - Year 1 / Semester B / Introduction to Website Building / summaries / web-development-course-summary.md

SEO CHEAT SHEET - Year 1 / Semester B / Introduction to Website Building / SEO CHEAT SHEET.pdf

Heuristic rules, lesson 4 - Year 1 / Semester A / User Centered Design / summaries / רועיש 4 יללכ הקיטסיריוהה.docx

Design principles, lesson 3 - Year 1 / Semester A / User Centered Design / summaries / רועיש 3 תונורקע תוציעב.docx

Interface evaluation, lesson 5 - Year 1 / Semester A / User Centered Design / summaries / מירחאמ / סיקשממ תכרעה- רועיש 5.docx

Final single page application brief - Year 1 / Semester B / Introduction to Website Building / היצקילפא תיינב _סויס ליגרת כ־תיביטקארטניא Single Page Application.pdf

HTML5 and CSS3 cheat sheets - Year 1 / Semester B / Introduction to Website Building / HTML5 CHEAT SHEET.pdf; CSS3 CHEAT SHEET.pdf

מבוא לבניית אתרים — סיכום מלא של הקורס והפרויקט

סטודנט: אהד רייז

קורס: מבוא לבניית אתרים באינטרנט

טכנולוגיות מרכזיות: HTML5, CSS3 ו־JavaScript בסיסי בעזרת בינה מלאכותית

פרויקט מסכם: סימולטור מונדיאל 2026 — משחק ניחוש לשלבי הנוקאאוט

תאריך עדכון הסיכום: 18 באוגוסט 2026

תוכן עניינים

1. מטרות הקורס ותמונת המצב הכללית
2. כיצד האינטרנט ואתר אינטרנט עובדים
3. כלי העבודה ותהליך הפיתוח
4. מבנה בסיסי של מסמך HTML5
5. החלק `head` ומטא־דאטא
6. תגיות סמנטיות ומבנה העמוד
7. כותרות, פסקאות ועיצוב סמנטי של טקסט
8. קישורים וניווט
9. רשימות
10. תמונות, `figure` ו־`figcaption`
11. טבלאות
12. מדיה: וידאו, אודיו ו־`iframe`
13. טפסים ופקדי קלט
14. יסודות CSS3
15. סלקטורים, מחלקות ומזהים

16. מפל העיצוב וסדרי קדימות
17. צבעים, פונטים וטקסט
18. מודל הקופסה
19. תצוגה ומיקום אלמנטים
20. Flexbox, Grid ורספונסיביות
21. נגישות
22. SEO ו־GEO
23. קוד נקי, תיעוד וולידציה
24. עבודה מושכלת עם בינה מלאכותית
25. JavaScript בפרויקט המסכם
26. תהליך בניית פרויקט הסיום
27. פרסום ב־Netlify ושילוב YouTube
28. טעויות נפוצות ורשימת בדיקות
29. דף עזר מהיר
30. מקורות הקורס שעליהם מבוסס הסיכום

---

1. מטרות הקורס ותמונת המצב הכללית

הקורס עסק בבניית אתרי אינטרנט באמצעות שתי שפות הליבה של צד הלקוח:

- **HTML5** — הגדרת המבנה, המשמעות והתוכן של הדף.
- **CSS3** — הגדרת המראה, הפריסה, הצבעים, הטיפוגרפיה וההתאמה למסכים שונים.

בפרויקט הסיום שולב גם **JavaScript**, אף על פי שהוא לא נלמד לעומק במסגרת הקורס. מטרתו הייתה להפוך אתר סטטי לאפליקציה אינטראקטיבית שמגיבה לפעולות המשתמש.

תוצרי הלמידה המרכזיים

בסיום הקורס למדנו:

- להבין את התחביר והמבנה של HTML5 ו־CSS3.
- ליצור מסמך HTML תקין ומאורגן.
- להשתמש בתגיות סמנטיות בהתאם למשמעות התוכן.
- ליצור כותרות, טקסטים, קישורים, רשימות, תמונות, טבלאות, מדיה וטפסים.
- לעצב אלמנטים באמצעות CSS ולא באמצעות מאפייני עיצוב בתוך HTML.
- להשתמש במודל הקופסה, בסלקטורים, במחלקות ובמזהים.
- למקם אלמנטים וליצור פריסה רספונסיבית.
- לשלב עקרונות נגישות, SEO ו־GEO כבר בזמן כתיבת הקוד.
- לכתוב קוד מסודר, קריא, מתועד וקל לתחזוקה.
- להשתמש בבינה מלאכותית ככלי עזר, בלי לוותר על הבנה, בדיקה ואחריות אישית.

שלוש השכבות של אפליקציית אינטרנט

שכבה	תפקיד	דוגמה מהפרויקט
HTML	מבנה ותוכן	טופס, אזורי תוכן, כפתורים, דיאלוגים ולוח הנוקאאוט
CSS	עיצוב ופריסה	צבעים, Grid, Flexbox, מצב מנצחת בירוק והתאמה למובייל
JavaScript	התנהגות ולוגיקה	בחירת מנצחות, מעבר בין שלבים, שמירה מקומית ופתיחת דיאלוגים

2. כיצד האינטרנט ואתר אינטרנט עובדים

Front-end לעומת Back-end

Front-end — צד הלקוח

זהו כל מה שהמשתמש רואה ומפעיל בדפדפן:

- תוכן ומבנה באמצעות HTML.
- עיצוב ופריסה באמצעות CSS.
- אינטראקטיביות באמצעות JavaScript.

Back-end — צד השרת

זהו החלק שפועל מאחורי הקלעים:

- לוגיקה עסקית.
- מסדי נתונים.
- אימות משתמשים.
- עיבוד נתונים.
- תקשורת עם שירותים אחרים.

בפרויקט שלנו לא נבנה שרת. כל האפליקציה פעלה בדפדפן ושמרה מידע באמצעות `localStorage` .

התהליך שמתרחש כשנכנסים לאתר

1. המשתמש מקליד כתובת URL.
2. הדפדפן פונה ל־DNS כדי לתרגם את שם הדומיין לכתובת IP.
3. הדפדפן שולח בקשת HTTP או HTTPS לשרת.
4. השרת מחזיר את קובצי האתר.
5. הדפדפן קורא את ה־HTML, טוען CSS, תמונות, פונטים ומדיה, ומפעיל JavaScript.
6. התוצאה מוצגת למשתמש.

מושגים חשובים

- **URL** — כתובת של משאב ברשת.
- **Domain** — שם האתר, לדוגמה `example.com` .
- **HTTP/HTTPS** — פרוטוקול להעברת מידע. HTTPS מוסיף הצפנה.
- **DNS** — מערכת שמתרגמת שם דומיין לכתובת IP.
- **Client** — הדפדפן או המכשיר שמבקש מידע.

- **Server** — המחשב שמחזיק ומחזיר את קובצי האתר.
- **W3C ו־WHATWG** — גופים העוסקים בתקנים ובמפרטי Web.
- **Browser** — תוכנה שמפרשת ומציגה HTML, CSS ו־JavaScript.

---

3. כלי העבודה ותהליך הפיתוח

עורך קוד

קוד כותבים בעורך טקסט ייעודי, כגון Visual Studio Code או Notepad++. עורך קוד עוזר באמצעות:

- צביעת תחביר.
- השלמה אוטומטית.
- סידור קבצים ותיקיות.
- איתור חלק מהשגיאות.
- תוספים, למשל Live Server.

תהליך עבודה מומלץ

1. מגדירים את מטרת האתר ואת קהל היעד.
2. מתכננים את מבנה התוכן והכותרות.
3. כותבים שלד HTML סמנטי.
4. מוסיפים תוכן, קישורים, רשימות ומדיה.
5. כותבים CSS מסודר לפי אזורים.
6. בודקים במסכים וברוחבים שונים.
7. מוסיפים אינטראקטיביות רק לאחר שהמבנה ברור.
8. בודקים נגישות, SEO, קישורים ומדיה.
9. מבצעים ולידציה וניקוי קוד.
10. מפרסמים ובודקים שוב בכתובת האמיתית.

בדיקה מקומית נכונה

אפשר לפתוח קובץ HTML ישירות, אך חלק מהשירותים החיצוניים דורשים אתר שמוגש דרך HTTP. לכן מומלץ להשתמש ב־Live Server או בשרת מקומי:

```
py -m http.server 8000
```

ואז לפתוח:

```
http://localhost:8000
```

4. מבנה בסיסי של מסמך HTML5

```
<!DOCTYPE html>
<html lang="he" dir="rtl">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="תיאור קצר וברור של תוכן הדף">
    <title>כותרת הדף</title>
</head>
<body>
    <header>
        <h1>כותרת ראשית</h1>
    </header>

    <main>
        <section>
            <h2>כותרת האזור</h2>
            <p>תוכן האזור.</p>
        </section>
    </main>

    <footer>
        <p>מידע תחתון.</p>
    </footer>
</body>
</html>
```

הסבר המבנה

- `<!DOCTYPE html>` מודיע לדפדפן שמדובר ב־HTML5.
- `<html>` עוטף את כל המסמך.
- `lang="he"` מגדיר עברית כשפת המסמך.
- `dir="rtl"` מגדיר כיוון כתיבה מימין לשמאל.
- `<head>` מכיל הגדרות ומטא־דאטא שאינם התוכן המרכזי המוצג.
- `<body>` מכיל את כל התוכן המוצג והאינטראקטיבי.

תגיות ריקות

תגיות ריקות אינן עוטפות תוכן ולכן אין להן תגית סוגרת:

- `<meta>`
- `<link>`
- `<img>`
- `<input>`

- `<br>`
- `<hr>`

קינון נכון — LIFO

התגית האחרונה שנפתחה צריכה להיות הראשונה שנסגרת:

```
<p><strong>טקסט חשוב</strong></p>
```

קינון שגוי:

```
<p><strong>טקסט חשוב</p></strong>
```

---

5. החלק `head` ומטא־דאטא

ההבדל בין `head`, `header` ו־`title` ו־`h1`

אלמנט	תפקיד
`<head>`	הגדרות המסמך, מטא־דאטא, קישורים לקבצים ועיצוב פנימי
`<header>`	אזור עליון סמנטי בתוך התוכן המוצג
`<title>`	שם הדף בכרטיסיית הדפדפן ובתוצאות חיפוש
`<h1>`	הכותרת הראשית של תוכן הדף

מטא־דאטא מרכזי

קידוד תווים

```
<meta charset="UTF-8">
```

מאפשר הצגה תקינה של עברית, אנגלית, סימנים ואימוג'י.

רספונסיביות

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

גורם לדף להשתמש ברוחב האמיתי של המכשיר.

תיאור הדף

```
<meta name="description" content="2026 סימולטור אינטראקטיבי לשלבי הנוקאאוט של מונדיאל">
```

התיאור מסביר למנועי חיפוש ולכלי AI מה מטרת הדף. הוא עשוי להופיע בתוצאות חיפוש.

מחבר

```
<meta name="author" content="Ohad Raiz">
```

מציין מי כתב את התוכן.

כותרת הדף

```
<title>2026 סימולטור מונדיאל</title>
```

הכותרת צריכה להיות תיאורית, ייחודית וקצרה יחסית.

קישור למשאב חיצוני

```
<link
    href="https://fonts.googleapis.com/css2?family=Heebo:wght@400;700;900&display=swap"
    rel="stylesheet"
>
```

`link` יכול לטעון קובץ CSS, פונט או משאב חיצוני אחר.

CSS פנימי

```
<style>
    body {
        font-family: Arial, sans-serif;
    }
</style>
```

בתעשייה לרוב מעדיפים קובץ CSS חיצוני. בפרויקט הסיום של הקורס הייתה דרישה מיוחדת שכל ה־CSS יהיה בתוך `style` ב־`head`.

---

6. תגיות סמנטיות ומבנה העמוד

סמנטיקה היא שימוש באלמנט לפי משמעות התוכן ולא רק לפי המראה הרצוי.

תגיות המבנה המרכזיות

תגית	תפקיד	שימוש בפרויקט
`<header>`	אזור פתיחה של דף או אזור	שם האתר וניווט; כותרות פנימיות
`<nav>`	אזור ניווט	קישורים לחלקי העמוד
`<main>`	התוכן הייחודי המרכזי	כל האפליקציה וההסברים
`<section>`	אזור נושאי	סימולטור, הוראות, מדיה, אודות ושאלות
`<article>`	יחידת תוכן עצמאית	כרטיסי תוכן וכרטיסי משחק
`<aside>`	תוכן משלים או צדדי	הערת מקור ופינת הזוכה האמיתית
`<footer>`	מידע מסכם ותחתון	יוצר הפרויקט וקישור לראש העמוד

`section` לעומת `article` לעומת `aside`

- משתמשים ב־`section` כאשר מחלקים את הדף לנושא או אזור בעל כותרת.
- משתמשים ב־`article` כאשר התוכן יכול לעמוד כיחידה עצמאית ולהיות מובן בפני עצמו.
- משתמשים ב־`aside` כאשר התוכן משלים את התוכן המרכזי אך אינו חלק הכרחי מהרצף שלו.

`div` ו־`span`

- `div` הוא מיכל בלוק ללא משמעות סמנטית.
- `span` הוא מיכל פנימי ללא משמעות סמנטית.

בתעשייה מותר להשתמש בהם כשאין אלמנט סמנטי מתאים. בפרויקט הסיום נאסר להשתמש בהם, כדי לעודד בחירה מודעת בתגיות סמנטיות.

היררכיית כותרות

לפי כללי הקורס:

- משתמשים ב־`h1` פעם אחת עבור הנושא הראשי.
- `h2` מחלק את הדף לנושאים מרכזיים.
- `h3` משמש כתת־נושא בתוך `h2`.

- לא בוחרים כותרת רק לפי גודל ברירת המחדל שלה.
- לא מדלגים על רמות ללא סיבה מבנית.

---

7. כותרות, פסקאות ועיצוב סמנטי של טקסט

אלמנטים בסיסיים

- `<p>` — פסקה.
- `<br>` — ירידת שורה בתוך אותו רצף תוכן.
- `<hr>` — הפרדה נושאית.
- `<pre>` — טקסט ששומר על רווחים וירידות שורה.

עיצוב חזותי לעומת משמעות סמנטית

חזותי בלבד	סמנטי	משמעות
`<b>`	`<strong>`	חשיבות חזקה
`<i>`	`<em>`	הדגשה סמנטית

עדיף להשתמש ב־`strong` וב־`em` כאשר יש משמעות, ואת המראה עצמו לקבוע ב־CSS.

אלמנטים נוספים

- `<mark>` — סימון מודגש, בדרך כלל בצבע.
- `<sup>` — כתב עילי.
- `<sub>` — כתב תחתי.
- `<small>` — טקסט משני קטן.
- `<del>` — תוכן שנמחק או בוטל.
- `<ins>` — תוכן שנוסף.
- `<dfn>` — הגדרת מונח.
- `<abbr>` — קיצור או ראשי תיבות.
- `<time>` — תאריך או שעה בפורמט שמכונות יכולות להבין.
- `<address>` — פרטי קשר של מחבר או ארגון.
- `<cite>` — שם של יצירה.
- `<q>` — ציטוט קצר.
- `<blockquote>` — ציטוט ארוך.

דוגמה ל־ `time`

```
<p>
    עודכן לאחרונה:
    <time datetime="2026-08-18">18 באוגוסט 2026</time>
</p>
```

`details` ו־ `summary`

```
<details>
    <summary>מהו HTML?</summary>
    <p>HTML היא שפת סימון להגדרת מבנה ותוכן.</p>
</details>
```

הם מתאימים במיוחד לאזור שאלות ותשובות, משום שהדפדפן מספק פתיחה וסגירה ללא JavaScript נוסף.

---

8. קישורים וניווט

קישור חיצוני

```
<a
    href="https://www.example.com"
    target="_blank"
    rel="noopener noreferrer"
>
                                                    מעבר לאתר לדוגמה
</a>
```

- `href` מגדיר את היעד.
- `target="_blank"` פותח לשונית חדשה.
- `rel="noopener noreferrer"` מגן על הדף המקורי ומצמצם העברת מידע מיותרת.

קישור פנימי לקובץ אחר

```
<a href="about.html">אודות</a>
```

זהו נתיב יחסי בתוך אותו אתר.

קישור לעוגן באותו עמוד

```
<a href="#instructions">הוראות</a>

<section id="instructions">
    <h2>הוראות</h2>
</section>
```

הערך אחרי `#` חייב להתאים ל־`id` של היעד.

תמונה כקישור

```
<a href="gallery.html">
    <img src="gallery.jpg" alt="מעבר לגלריית התמונות">
</a>
```

ה־`alt` צריך לתאר גם את מטרת הקישור.

טקסט קישור טוב

עדיף:

```
<a href="instructions.html">קריאת הוראות השימוש</a>
```

פחות טוב:

```
<a href="instructions.html">לחצו כאן</a>
```

טקסט תיאורי עוזר למשתמשים, לקוראי מסך, למנועי חיפוש ולמודלי שפה.

---

9. רשימות

רשימה לא ממוספרת

מתאימה לפריטים שאין חשיבות לסדרם:

```
<ul>
    <li>HTML</li>
    <li>CSS</li>
    <li>JavaScript</li>
</ul>
```

מתאימה גם לתפריטי ניווט.

רשימה ממוספרת

מתאימה לשלבים ולהוראות:

```
<ol>
    <li>פותחים קובץ.</li>
    <li>כותבים HTML.</li>
    <li>בודקים בדפדפן.</li>
</ol>
```

מאפיינים שנלמדו:

- `start` — המספר הראשון.
- `value` — מספר מיוחד לפריט מסוים.
- `type` — סוג המספור.
- `reversed` — ספירה לאחור.

רשימת הגדרות

```
<dl>
    <dt>HTML</dt>
    <dd>שפת סימון למבנה ולתוכן.</dd>

    <dt>CSS</dt>
    <dd>שפה לעיצוב ולפריסה.</dd>
</dl>
```

- `dl` עוטף את הרשימה.
- `dt` הוא המונח.

- `dd` הוא ההסבר.

---

10. תמונות, `figure` ו־`figcaption`

תמונה בסיסית

```
<img
    src="world-cup-bracket.jpg"
    alt="תרשים של משחקי שלב הנוקאאוט"
>
```

מאפיינים חשובים

- `src` — כתובת או נתיב התמונה.
- `alt` — תיאור חלופי נגיש.
- `title` — מידע משלים בריחוף; אינו תחליף ל־`alt`.
- `loading="lazy"` — דחיית טעינה עד שהתמונה מתקרבת למסך.

`figure` ו־`figcaption`

```
<figure>
    <img src="stadium.jpg" alt="אצטדיון כדורגל מלא בקהל">
    <figcaption>אצטדיון במהלך משחק במונדיאל.</figcaption>
</figure>
```

`figure` מאגד המחשה עצמאית עם ההסבר שלה. הוא יכול להכיל תמונה, תרשים, קוד, ציטוט או המחשה אחרת.

פורמטים שנלמדו

- **JPG/JPEG** — מתאים לצילומים, אינו תומך בשקיפות מלאה.
- **PNG** — איכות גבוהה ושקיפות, בדרך כלל קובץ כבד יותר.
- **GIF** — תומך באנימציה ובמספר צבעים מוגבל.

כדאי לבחור פורמט מתאים ולצמצם משקל קבצים כדי לשפר ביצועים.

תמונה רספונסיבית בסיסית

```
img {
    max-width: 100%;
    height: auto;
}
```

כך התמונה אינה חורגת מהמיכל ושומרת על יחס הממדים.

11. טבלאות

טבלה מיועדת לנתונים טבלאיים בלבד, לא לפריסת העמוד.

```
<table>
    <caption>התקדמות לפי שלב</caption>
    <thead>
        <tr>
            <th scope="col">שלב</th>
            <th scope="col">מספר משתקים</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>שמינית גמר</td>
            <td>8</td>
        </tr>
        <tr>
            <td>רבע גמר</td>
            <td>4</td>
        </tr>
    </tbody>
</table>
```

תפקידי התגיות

- `table` — הטבלה כולה.
- `caption` — כותרת הטבלה, מיד לאחר `table`.
- `thead` — אזור הכותרות.
- `tbody` — גוף הנתונים.
- `tfoot` — שורות סיכום.
- `tr` — שורה.
- `th` — תא כותרת.
- `td` — תא נתונים.
- `scope` — מסביר אם הכותרת מתייחסת לעמודה או לשורה.

כללים חשובים

- לכל השורות צריך להיות מבנה עקבי.
- משתמשים ב־`th` לכותרות ולא רק לצורך הדגשה.
- מעצבים את הטבלה ב־CSS.
- `border-collapse: collapse` מונע גבולות כפולים.

12. מדיה: וידאו, אודיו ו־`iframe`

וידאו מקומי

```
<video controls poster="poster.jpg">
    <source src="explanation.mp4" type="video/mp4">
                                                    הדפדפן אינו תומך בווידאו.
</video>
```

אודיו

```
<audio controls preload="metadata">
    <source src="sound.ogg" type="audio/ogg">
                                                    הדפדפן אינו תומך באודיו.
</audio>
```

מאפייני מדיה מרכזיים

- `controls` — פקדי הפעלה.
- `autoplay` — הפעלה אוטומטית; דפדפנים עשויים לחסום אותה.
- `muted` — השתקה.
- `loop` — חזרה.
- `poster` — תמונה מקדימה לווידאו.
- `preload` — רמת טעינה מוקדמת.

הטמעת YouTube

```
<iframe
    src="https://www.youtube.com/embed/VIDEO_ID"
    title="תיאור ברור של הסרטון"
    loading="lazy"
    referrerpolicy="strict-origin-when-cross-origin"
    allow="accelerometer; clipboard-write; encrypted-media; gyroscope; picture-in-picture; web-
share"
    allowfullscreen
></iframe>
```

חשוב להשתמש בכתובת `embed` ולא בכתובת הצפייה הרגילה. סרטון YouTube עשוי לא לעבוד כאשר פותחים את הדף דרך `file:///` , משום שהנגן דורש `HTTP Referer` . לכן בודקים דרך Live Server, `localhost` או אתר שפורסם.

---

13. טפסים ופקדי קלט

טופס מאפשר תקשורת בין המשתמש לאתר. בקורס התמקדנו בבניית הטופס בצד הלקוח ולא בשליחת נתונים לשרת.

טופס בסיסי

```
<form>
    <label for="full-name">שם מלא</label>
    <input
        id="full-name"
        name="full-name"
        type="text"
        required
    >

    <label for="favorite-team">נבחרת אהודה</label>
    <select id="favorite-team" name="favorite-team" required>
        <option value="">בחירת נבחרת</option>
        <option value="spain">ספרד</option>
        <option value="argentina">ארגנטינה</option>
    </select>

    <button type="submit">שמירה</button>
</form>
```

`label`

`label` מסביר מה נדרש בשדה. אפשר לקשר אותו בשתי דרכים:

1. באמצעות `for` שמצביע ל־`id`.
2. באמצעות קינון השדה בתוך ה־`label`.

קישור נכון מאפשר גם ללחוץ על הטקסט כדי להתמקד בשדה.

`id` לעומת `name`

- `id` מזהה אלמנט ייחודי בדף ומשמש CSS, JavaScript, קישורי עוגן ו־`label`.
- `name` הוא שם הנתון בזמן שליחת טופס.

סוגי `input` מרכזיים

- `text` — טקסט קצר.
- `email` — כתובת דואר אלקטרוני עם בדיקה בסיסית.
- `tel` — טלפון.
- `number` — מספר.

- `date` — תאריך.
- `color` — בחירת צבע.
- `range` — טווח מספרי.
- `radio` — בחירה אחת מתוך קבוצה.
- `checkbox` — בחירה של כמה אפשרויות.
- `password` — סיסמה מוסתרת.
- `submit` — שליחת הטופס.

מאפיינים שימושיים

- `required` — שדה חובה.
- `placeholder` — דוגמה זמנית; אינו תחליף ל־`label`.
- `min`, `max`, `step` — מגבלות מספריות.
- `maxlength` — אורך טקסט מרבי.
- `checked` — בחירה התחלתית.
- `autocomplete` — הצעות מהדפדפן.

`textarea`

```
<label for="message">הודעה</label>
<textarea id="message" name="message" rows="6"></textarea>
```

מתאים לטקסט ארוך ורב־שורות.

`fieldset` ו־`legend`

```
<fieldset>
    <legend>דרך התקשרות מועדפת</legend>

    <label>
        <input type="radio" name="contact" value="email">
דואר אלקטרוני
    </label>

    <label>
        <input type="radio" name="contact" value="phone">
טלפון
    </label>
</fieldset>
```

הם מאגדים קבוצת שדות קשורים באופן סמנטי ונגיש.

`button`

חשוב להגדיר סוג מפורש:

- `type="submit"` — שולח טופס.
- `type="reset"` — מאפס שדות טופס בסיסיים.
- `type="button"` — מפעיל פעולה שאינה שליחה.

בפרויקט השתמשנו בכפתורי `button` לבחירת מנצחות, פתיחת דיאלוגים ואיפוס הסימולציה.

`output` ו־`progress`

```
<output id="status" aria-live="polite">הטופס עדיין לא נשמר.</output>

<progress id="progress" value="0" max="31">
                                                                    מתוך 31 0
</progress>
```

- `output` מייצג פלט של פעולה או חישוב.
- `progress` מייצג התקדמות במשימה בעלת ערך מרבי.

---

14. יסודות CSS3

CSS מפריד בין מבנה ותוכן לבין עיצוב.

תחביר בסיסי

```
selector {
    property: value;
}
```

דוגמה:

```
p {
    color: #14213d;
    font-size: 1rem;
}
```

- `p` הוא הסלקטור.
- `color` הוא המאפיין.
- `#14213d` הוא הערך.
- כל הכרזה מסתיימת בנקודה־פסיק.

שלוש דרכי שילוב CSS

קובץ חיצוני

```
<link href="styles.css" rel="stylesheet">
```

זוהי הדרך המועדפת ברוב הפרויקטים המקצועיים.

CSS פנימי

```
<style>
    body {
        background-color: #ffffff;
    }
</style>
```

בפרויקט הסיום זו הייתה דרישת המטלה.

Inline

```
<p style="color: red;">טקסט</p>
```

עדיף להימנע משום שהוא מערבב מבנה ועיצוב, יוצר כפילויות ומקשה על תחזוקה.

הערות CSS

```
/* עיצוב אזור הניווט */
```

---

15. סלקטורים, מחלקות ומזהים

סלקטור אלמנט

```
h2 {
    color: #003f87;
}
```

חל על כל תגיות `h2` .

מחלקה — `class`

:HTML

```
<button class="button-primary">התחלה</button>
```

:CSS

```
.button-primary {
    color: #ffffff;
    background-color: #0057b8;
}
```

מחלקה מיועדת לעיצוב חוזר. לפני שם מחלקה ב־CSS כותבים נקודה.

מזהה — `id`

:HTML

```
<section id="simulator">
```

:CSS

```
#simulator {
    display: grid;
}
```

`id` אמור להיות ייחודי. לפניו ב־CSS כותבים `#` .

סלקטור צאצא

```
.site-nav a {
    text-decoration: none;
}
```

בוחר קישורים שנמצאים בתוך `.site-nav` .

סלקטור ילד ישיר

```
#simulator > * {
    grid-column: 1 / -1;
}
```

בוחר רק ילדים ישירים של `#simulator` .

סלקטור מאפיין

```
.team-button[aria-pressed="true"] {
    background-color: #166534;
}
```

בפרויקט הוא שימש לסימון מנצחת בירוק לפי מצב נגיש.

פסאודו־מחלקות

```
button:hover {
    background-color: #003f87;
}

button:focus-visible {
    outline: 3px solid #ffcc00;
}
```

- `:hover` — מעבר עכבר.
- `:focus-visible` — מיקוד מקלדת נראה.
- `:first-child` , `:last-child` , `:nth-child()` — בחירה לפי מיקום.

פסאודו־אלמנט

```
.result-dialog article::before {
    content: "";
    position: absolute;
}
```

`::before` יוצר שכבה חזותית בלי להוסיף אלמנט HTML נוסף. בפרויקט הוא שימש לרקע שקוף של דגל ספרד.

---

16. מפל העיצוב וסדרי קדימות

CSS הוא Cascading Style Sheets — גיליונות סגנון מדורגים.

כאשר כמה כללים משפיעים על אותו מאפיין, הדפדפן מתחשב ב:

1. חשיבות המקור והאם קיים `!important` .
2. רמת הספציפיות של הסלקטור.
3. סדר הופעת הכללים בקובץ כאשר הספציפיות זהה.

באופן כללי:

```
inline style > id > סלקטור אלמנט > מחלקה/מאפיין
```

עדיף לא לפתור בעיות באמצעות `!important` , אלא לשמור על מבנה סלקטורים פשוט ועקבי.

ירושה

מאפיינים מסוימים עוברים מהורה לילדים, לדוגמה:

- `color`
- `font-family`
- `line-height`

מאפייני פריסה, גבול ושוליים בדרך כלל אינם עוברים אוטומטית.

---

17. צבעים, פונטים וטקסט

צבעים

אפשר להגדיר צבע בכמה דרכים:

```
color: blue;
color: #0057b8;
color: rgb(0, 87, 184);
color: rgba(0, 87, 184, 0.5);
```

בפרויקט השתמשנו בעיקר בקודי HEX ובּ־RGBA לצללים ולשקיפות.

פלטת צבעים

כדאי לבחור פלטה עקבית:

- צבע ראשי.
- צבע משני.
- צבע רקע.
- צבע טקסט.
- צבע אזהרה.
- צבעי מצב, למשל ירוק להצלחה ואפור להדחה.

פונטים

```
body {
    font-family: "Heebo", "Segoe UI", sans-serif;
}
```

הדפדפן מנסה את הפונט הראשון. אם הוא אינו זמין, הוא עובר לבא אחריו.

מאפייני טקסט מרכזיים

- `font-family` — משפחת פונטים.
- `font-size` — גודל.
- `font-weight` — עובי, למשל `400` , `700` , `900` .
- `font-style` — רגיל או נטוי.
- `line-height` — מרווח בין שורות.
- `letter-spacing` — מרווח בין אותיות.

- `text-align` — יישור.
- `text-decoration` — קו תחתון, מחיקה וכדומה.
- `text-shadow` — צל לטקסט.

יחידות מידה שימושיות

- `px` — יחידה קבועה יחסית.
- `%` — יחס למיכל.
- `em` — יחס לגודל הפונט של האלמנט או ההורה.
- `rem` — יחס לגודל הפונט של מסמך השורש.
- `vw` — אחוז מרוחב המסך.
- `vh` — אחוז מגובה המסך.

---

18. מודל הקופסה

כל אלמנט חזותי בדף מתנהג כקופסה הבנויה מארבע שכבות:

1. **Content** — התוכן.
2. **Padding** — רווח פנימי בין התוכן לגבול.
3. **Border** — המסגרת.
4. **Margin** — רווח חיצוני בין האלמנט לסביבתו.

```
.card {
    width: 20rem;
    margin: 1rem;
    padding: 1.5rem;
    border: 2px solid #0057b8;
}
```

סדר הערכים בהגדרה מקוצרת

ארבעה ערכים:

```
padding: top right bottom left;
```

שלושה ערכים:

```
padding: top right-left bottom;
```

שני ערכים:

```
padding: top-bottom right-left;
```

ערך אחד חל על כל הצדדים.

`box-sizing`

ברירת המחדל היא `content-box`, שבה `padding` ו־`border` מתווספים לרוחב.

```
* {
    box-sizing: border-box;
}
```

ב־`border-box`, הרוחב כולל גם את ה־padding וה־border. כך החישובים צפויים ופשוטים יותר.

19. תצוגה ומיקום אלמנטים

`display`

Block

```
display: block;
```

האלמנט מתחיל בשורה חדשה ותופס בדרך כלל את הרוחב הזמין.

Inline

```
display: inline;
```

האלמנט נשאר בתוך רצף הטקסט ולא מקבל בדרך כלל רוחב וגובה רגילים.

Inline-block

```
display: inline-block;
```

האלמנטים יכולים להופיע זה לצד זה, ועדיין לקבל רוחב וגובה.

None

```
display: none;
```

האלמנט מוסר מהתצוגה ומהפריסה.

`float`

נלמד ככלי להצגת תמונה וטקסט זה לצד זה. בפריסה מודרנית בדרך כלל מעדיפים Flexbox או Grid.

`position`

ערך	משמעות
`static`	ברירת המחדל לפי סדר הקוד
`relative`	נשאר במקום הטבעי ויכול לזוז יחסית לעצמו; משמש גם עוגן ל־absolute
`absolute`	יוצא מהזרימה וממוקם יחסית לאב ממוקם
`fixed`	נשאר במקום קבוע ביחס למסך
`sticky`	מתנהג רגיל עד לנקודת גלילה ואז נצמד

המיקום נקבע באמצעות `top` , `right` , `bottom` ו־ `left` .

`z-index`

מגדיר איזו שכבה תוצג מעל האחרת כאשר אלמנטים חופפים. הוא רלוונטי בעיקר לאלמנטים ממוקמים.

בפרויקט:

- הכותרת הראשית נשארה נעוצה למעלה.
- סרגל ההתקדמות נשאר גלוי בזמן מילוי הבארקט.
- כותרות שלבי הטורניר נשארו בראש אזור הגלילה.

---

20. Flexbox, Grid ורספונסיביות

Flexbox

מתאים לפריסה בכיוון מרכזי אחד — שורה או עמודה.

```
.navigation-list {
    display: flex;
    align-items: center;
    justify-content: space-between;
    flex-wrap: wrap;
    gap: 1rem;
}
```

מאפיינים מרכזיים:

- `flex-direction`
- `justify-content`
- `align-items`
- `flex-wrap`
- `gap`

CSS Grid

מתאים לפריסה דו־ממדית של שורות ועמודות.

```
.cards {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 1.5rem;
}
```

בפרויקט Grid שימש עבור:

- טופס המשתמש.
- חמש עמודות שלבי הנוקאאוט.
- כרטיסי ההוראות והאודות.
- שני הכרטיסים שבסוף הסימולטור.

רספונסיביות

אתר רספונסיבי מתאים את עצמו לרוחבי מסך שונים.

```
@media (max-width: 800px) {
    .cards {
        grid-template-columns: 1fr;
    }
}
```

עקרונות:

- תמונות ומדיה לא חורגות מהמיכל.
- טקסט נשאר קריא.
- כפתורים גדולים מספיק למגע.
- עמודות הופכות לעמודה אחת במסך צר.
- ניווט יכול לעבור לשורה נוספת.
- תוכן רחב במיוחד יכול לקבל גלילה אופקית מבוקרת.

Bootstrap

נלמד בקצרה ככלי עיצוב מוכן:

- מבוסס על מערכת של 12 עמודות.
- כולל מחלקות מוכנות לכפתורים, טקסט, קופסאות ופריסה.
- מסייע ברספונסיביות ובתאימות לדפדפנים.

עם זאת, בפרויקט הסיום נאסר להשתמש ב־Framework שמחליף את כתיבת ה־HTML וה־CSS הבסיסיים, ולכן העיצוב נכתב ידנית.

---

21. נגישות

נגישות מאפשרת לאנשים עם מגבלות ראייה, שמיעה, תנועה או הבנה להשתמש באתר.

עקרונות מרכזיים

- מבנה HTML סמנטי.
- היררכיית כותרות עקבית.
- ניווט מקלדת מלא.
- טקסט קריא וניגודיות טובה.
- כפתורים וקישורים בעלי שמות ברורים.
- `alt` מתאים לתמונות.
- `label` לכל שדה טופס.
- הודעות מצב נגישות.
- הימנעות מהסתמכות על צבע בלבד.
- כתוביות או תמלול למדיה מדוברת.

`aria-label`

מוסיף שם נגיש כאשר אין טקסט גלוי מתאים:

```
<nav aria-label="ניווט ראשי">
```

הערך עצמו מוקרא; הוא אינו מצביע ל־ `id` .

`aria-labelledby`

מפנה לכותרת קיימת באמצעות `id` :

```
<section aria-labelledby="instructions-title">
    <h2 id="instructions-title">הוראות שימוש</h2>
</section>
```

`aria-pressed`

מתאים לכפתור בעל מצב לחוץ או לא לחוץ:

```
<button type="button" aria-pressed="false">טפרד</button>
```

בפרויקט JavaScript עדכן את הערך, ו־CSS צבע את הכפתור שנבחר בירוק.

`aria-live`

```
<output aria-live="polite">הבחירה נשמרה.</output>
```

מאפשר לקורא מסך להודיע על טקסט דינמי בלי להעביר אליו מיד את המיקוד.

ניווט מקלדת

- `Tab` מעביר בין אלמנטים אינטראקטיביים.
- `Shift + Tab` חוזר לאחור.
- `Enter` או רווח מפעילים כפתור.
- `tabindex="0"` יכול להפוך אלמנט שאינו אינטראקטיבי לממוקד, אך יש להשתמש בזה בזהירות.

מיקוד נראה

```
button:focus-visible {
    outline: 3px solid #ffcc00;
    outline-offset: 2px;
}
```

אין להסיר `outline` בלי לספק חלופה ברורה.

ניגודיות

לפי WCAG, טקסט רגיל צריך בדרך כלל יחס ניגודיות של לפחות 4.5:1 מול הרקע.

---

22. SEO ו־GEO

SEO

Search Engine Optimization — התאמת האתר למנועי חיפוש.

GEO

Generative Engine Optimization — התאמת התוכן למערכות חיפוש ויצירת תשובות מבוססות בינה מלאכותית.

שלושת הקהלים

בבניית אתר מקצועי מתחשבים ב:

1. בני אדם, כולל אנשים עם מוגבלויות.
2. מנועי חיפוש מסורתיים.
3. מודלי שפה וכלי AI.

אותם עקרונות משרתים לרוב את שלושתם: מבנה סמנטי, כותרות מסודרות, תוכן ברור, מקורות אמינים ונגישות.

פעולות חשובות ב־`head`

- `title` תיאורי וייחודי.
- `meta description` ברור.
- `viewport` לרספונסיביות.
- `lang` לשפת המסמך.
- במידת הצורך `robots` ו־`canonical`.

הערה: `meta keywords` נלמד כחלק ממטא־דאטא, אך לפי חומר ה־SEO המודרני גוגל מתעלם ממנו לצורכי דירוג מאז 2009.

פעולות חשובות בתוכן

- `h1` ברור לנושא הראשי.
- חלוקה ל־`h2` ו־`h3`.
- פסקת פתיחה שמסבירה מיד את מטרת הדף.
- מילות מפתח בשפה טבעית, ללא דחיסה מלאכותית.
- תוכן מקורי, אמין ומעודכן.
- קישורים פנימיים וחיצוניים בעלי טקסט תיאורי.
- אזור About שמציג מי עומד מאחורי התוכן.

- אזור שאלות ותשובות.
- ציון מקורות ותאריכי עדכון.

מדיה ו־SEO

- שם קובץ תיאורי.
- `alt` מדויק.
- `figure` ו־`figcaption` להקשר.
- קבצים קלים לטעינה.
- כתוביות ותמלול לווידאו.

E-E-A-T

- **Experience** — ניסיון מעשי.
- **Expertise** — מומחיות.
- **Authoritativeness** — סמכותיות.
- **Trustworthiness** — אמינות.

פרטי מחבר, מקורות, תאריך עדכון ותוכן ברור מסייעים לבניית אמון.

SPA ואתגרי חיפוש

באפליקציה של עמוד אחד כל התוכן מרוכז במסמך אחד וחלקו משתנה באמצעות JavaScript. לכן חשוב במיוחד:

- להשאיר תוכן הסבר חשוב בתוך HTML ולא רק ליצור אותו דינמית.
- להשתמש באזורים סמנטיים.
- לתת כותרת לכל נושא.
- לכלול About ו־Q&A.
- לתת שמות ברורים לכפתורים ולהודעות.

---

23. קוד נקי, תיעוד וולידציה

קוד שעובד אינו בהכרח קוד נקי. קוד נקי צריך להיות מובן גם לבני אדם, קל לתחזוקה וקל לשינוי.

KISS

Keep It Simple — לבחור את הפתרון הפשוט והברור ביותר ולא להוסיף תחכום שאינו נחוץ.

DRY

Don't Repeat Yourself — להימנע מחזרות מיותרות. לדוגמה, בפרויקט שמות הנבחרות נשמרו במקום אחד ו־JavaScript השתמש בהם בטופס ובלוח המשחקים.

YAGNI

You Aren't Gonna Need It — לא לכתוב תכונות שאין להן דרישה ממשית כרגע.

שיטתיות

- סגנון הזחה אחיד.
- כל הכרזת CSS בשורה נפרדת.
- סדר הגיוני ועקבי של מאפיינים.
- שמות מחלקות ומשתנים ברורים.
- חלוקה לאזורים בעזרת הערות.
- אורך שורות סביר.
- הסרת קוד שלא נעשה בו שימוש.

שמות קבצים ומשתנים

לפי כללי הקורס:

- אותיות אנגליות קטנות.
- בלי רווחים.
- שמות שמתארים את התוכן.
- אפשר להשתמש במקף או בקו תחתון.

טוב:

```
world-cup-bracket.jpg
favoriteTeam
participantForm
```

פחות טוב:

```
1234.jpg
my file.html
sus.jpg
```

הערות

הערה טובה מסבירה את המבנה, הסיבה או החלטה שאינה ברורה מהקוד. אין צורך להסביר כל שורה ברורה.

```
<!-- טופס פתיחת הבארקט לאחר מילוי פרטי המשתמש
```

```
/* סימון הנבחרת שנבחרה כמנצחת */
```

ולידציה

מומלץ להשתמש ב־W3C Validator:

```
https://validator.w3.org/
```

אפשר לבדוק באמצעות:

- כתובת של אתר שפורסם.
- העלאת קובץ.
- הדבקת קוד.

מה מחפשים

- תגיות שלא נסגרו.
- קינון שגוי.
- מאפיינים לא חוקיים.
- `id` כפול.
- `alt` חסר.
- היררכיית כותרות לא תקינה.
- שגיאות מבנה בטבלאות ובטפסים.

---

24. עבודה מושכלת עם בינה מלאכותית

בינה מלאכותית יכולה לעזור ב:

- תכנון מבנה.
- יצירת שלד קוד.
- הסבר מושגים.
- הצעת עיצוב.
- איתור שגיאות.
- Debugging.
- שיפור נגישות ורספונסיביות.
- יצירת בדיקות ורשימות ביקורת.

עם זאת:

- AI עלול להמציא מידע או קוד.
- קוד יכול להיות תקין תחבירית אך שגוי לוגית.
- אסור למסור מידע רגיש ללא צורך.
- האחריות על התוצאה היא של הכותב.
- צריך להבין כל שורת קוד שמגישים.
- יש להסיר כפילויות וקוד לא רלוונטי.
- יש להתאים את הקוד לרמת הקורס ולהנחיות המרצה.

תהליך עבודה נכון עם AI

1. מנסחים דרישה ברורה וממוקדת.
2. מבקשים פתרון קטן ולא שינוי כללי מדי.
3. קוראים את ההסבר ולא רק מעתיקים.
4. בודקים שהפתרון מתאים ל-HTML הקיים.
5. בודקים בדפדפן.
6. בודקים שגיאות בקונסול.
7. מסירים קוד מיותר.
8. מוסיפים תיעוד במילים שלנו.

25. JavaScript בפרויקט המסכם

JavaScript לא היה נושא מרכזי בקורס, אך נדרש בפרויקט כדי ליצור אפליקציה אינטראקטיבית.

מטרות JavaScript בפרויקט

- יצירת רשימת הנבחרות בטופס.
- יצירת 31 כרטיסי משחק באופן דינמי.
- בחירת מנצחת בכל משחק.
- העלאת המנצחת לשלב הבא.
- ביטול או שינוי בחירה.
- ניקוי החלטות שתלויות בתוצאה קודמת.
- ספירת ההתקדמות.
- הצגת האלופה.
- שמירה ושחזור באמצעות `localStorage` .
- פתיחת דיאלוגים.
- הפעלת אודיו וצליל לחיצה.

משתנים ונתונים

```
const teams = {
    ESP: { name: "ספרד" },
    ARG: { name: "ארגנטינה" }
};

let applicationState = {
    participant: null,
    winners: {}
};
```

- `const` משמש להפניה שאינה אמורה להיות מוחלפת.
- `let` משמש לערך שיכול להשתנות.
- אובייקט מאגד מידע לפי מפתחות.
- מערך מאגד רשימת פריטים בסדר מסוים.

בחירת אלמנטים מה־DOM

```
const participantForm = document.querySelector("#participant-form");
const progressElement = document.querySelector("#tournament-progress");
```

ה־DOM הוא הייצוג של מסמך HTML בזיכרון הדפדפן.

אירועים

```
participantForm.addEventListener("submit", (event) => {
    event.preventDefault();
});
```

- `addEventListener` מחבר פעולה לאירוע.
- `submit` מתרחש בשליחת טופס.
- `click` מתרחש בלחיצה.
- `event.preventDefault()` מונע את פעולת ברירת המחדל, למשל רענון הדף.

יצירת אלמנטים דינמית

```
const option = document.createElement("option");
option.value = "ESP";
option.textContent = "ספרד";
select.append(option);
```

- `createElement` יוצר אלמנט.
- `textContent` מכניס טקסט בטוח.
- `append` מוסיף אותו למסמך.

`dataset`

:HTML

```
<button data-match-id="73" data-team-code="ESP">ספרד</button>
```

:JavaScript

```
const matchId = button.dataset.matchId;
const teamCode = button.dataset.teamCode;
```

מאפייני `data-*` מאפשרים להצמיד מידע מותאם לאלמנט.

שמירה מקומית

```
localStorage.setItem("bracket", JSON.stringify(applicationState));

const savedState = JSON.parse(localStorage.getItem("bracket"));
```

- `localStorage` שומר מחרוזות בדפדפן.
- `JSON.stringify` ממיר אובייקט למחרוזת.
- `JSON.parse` מחזיר את המחרוזת לאובייקט.

ציור מחדש — Rendering

במקום לשנות כל חלק ידנית, האפליקציה שמרה מצב מרכזי ואז הפעילה פונקציות ציור:

- `renderBracket()`
- `renderProgress()`
- `renderChampion()`

כך המסך נשאר מסונכרן עם הנתונים.

תלות בין משחקים

אם המשתמש משנה מנצחת במשחק מוקדם, ייתכן שהנבחרת כבר נבחרה בשלבים מאוחרים. לכן האפליקציה:

1. מוצאת את כל המשחקים שתלויים בתוצאה.
2. מציגה בקשת אישור אם קיימות הכרעות מאוחרות.
3. מוחקת רק את ההכרעות שאינן תקפות יותר.
4. מציירת מחדש את הלוח.

זה היה החלק הלוגי המורכב ביותר בפרויקט.

דיאלוגים

```
resultDialog.showModal();
```

`showModal()` פותח את `<dialog>` כחלון ממוקד וחוסם זמנית אינטראקציה עם הרקע.

Web Audio API

בפרויקט נוצר צליל לחיצה קצר באמצעות `AudioContext` , `Oscillator` ו־ `Gain` . זו דוגמה ליצירת צליל באמצעות JavaScript בלי להוסיף קובץ אודיו נוסף.

---

26. תהליך בניית פרויקט הסיום

דרישות מרכזיות

- קובץ `index.html` אחד.
- אפליקציית SPA אינטראקטיבית.
- HTML, CSS ו־JavaScript בסיסיים בלבד.
- ללא React, Vue, Angular או Framework אחר.
- מבנה סמנטי מלא.
- ללא `div` וללא `span` לצורך התרגיל.
- CSS בתוך `style` ב־`head`.
- JavaScript במקום אחד בסוף `body`.
- מסך פתיחה ורכיב אינטראקטיבי מרכזי.
- הוראות, About, Q&A וניווט פנימי.
- תמונה, וידאו, אודיו, קישורים ורשימות.
- נגישות, רספונסיביות, SEO ו־GEO.
- לפחות שני אלמנטים חדשים עם הערות הסבר.
- סרטון הסבר על האפליקציה והקוד.
- פרסום ב־Netlify.

הרעיון שנבחר

נבנה סימולטור נוקאאוט למונדיאל 2026:

- 32 נבחרות.
- 16 משחקים בשלב 32 האחרונות.
- 8 משחקי שמינית גמר.
- 4 משחקי רבע גמר.
- 2 משחקי חצי גמר.
- משחק גמר אחד.
- סך הכול 31 הכרעות.

מבנה האפליקציה

1. כותרת וניווט נעוץ.
2. מסך פתיחה עם הסבר ותמונת בארקט.
3. טופס שם ונבחרת אהודה.

4. סרגל התקדמות נעוץ.
5. לוח נוקאאוט אינטראקטיבי.
6. כרטיס האלופה שבחר המשתמש.
7. דיאלוג הזוכה האמיתית.
8. הוראות שימוש.
9. סרטון הסבר.
10. אזור אודות ורפלקציה.
11. שאלות ותשובות.
12. Footer עם מידע מסכם.

אלמנטים חדשים שנחקרו

- `select` ו־`option` — רשימת נבחרות.
- `output` — הודעות מצב ואלופה.
- `progress` — התקדמות מתוך 31.
- `details` ו־`summary` — שאלות ותשובות.
- `dialog` — תוצאה, חגיגה ואישור פעולה.
- `time` — תאריך בעל משמעות למכונות.

מצבי עיצוב מרכזיים

- הנבחרת האהודה — צהוב דהוי.
- המנצחת שנבחרה — ירוק.
- המודחת — אפור ושקיפות.
- שלב נוכחי — כותרת מודגשת.
- בארקט נעול — שקיפות עד מילוי הטופס.

החלטות חשובות

- שמירת נתוני הנבחרות במקום מרכזי אחד.
- יצירת משחקים ב־JavaScript במקום שכפול HTML.
- ניקוי בחירות תלויות בלבד.
- שמירת מצב בדפדפן.
- שימוש בכפתורים ולא בקישורים עבור פעולות.
- שימוש ב־ARIA למצבי בחירה ולהודעות.
- בדיקה דרך Netlify כדי לוודא שמדיה חיצונית נטענת.

27. פרסום ב־Netlify ושילוב YouTube

פרסום ידני ב־Netlify

1. מוודאים ש־`index.html` נמצא בשורש תיקיית הפרויקט.
2. נכנסים ל־Netlify Drop.
3. גוררים את תיקיית הפרויקט.
4. ממתינים למצב Published.
5. פותחים את כתובת `netlify.app` .
6. בודקים שוב את כל האפליקציה.
7. לאחר שינוי גוררים שוב את התיקייה לאזור Deploys של אותו פרויקט.

שילוב YouTube

1. מעלים את הסרטון ל־YouTube.
2. מגדירים Public או Unlisted, לא Private.
3. מסמנים Allow embedding.
4. בוחרים Share ואז Embed.
5. מעתיקים את כתובת ה־embed אל ה־`iframe` .
6. בודקים דרך `localhost` או Netlify, לא דרך `file:///` .

קישורים ציבוריים

- קישור שמתחיל ב־`https://app.netlify.com` מיועד לניהול ואינו קישור האתר לציבור.
- הקישור הציבורי מסתיים בדרך כלל ב־`netlify.app` .
- וידאו Unlisted פתוח לכל מי שיש לו את הקישור, אך אינו אמור להופיע בתוצאות חיפוש רגילות.

---

28. טעויות נפוצות ורשימת בדיקות

טעויות HTML נפוצות

- תגית פותחת ללא סוגרת.
- קינון שגוי.
- יותר מ־`h1` אחד לפי כללי הקורס.
- דילוג לא הגיוני בין רמות כותרת.
- `section` או `article` ללא כותרת לפי דרישות הקורס.
- `id` כפול.
- ערכי מאפיינים ללא מרכאות.
- שימוש ב־`style` בתוך HTML.
- `img` ללא `alt`.
- `target="_blank"` ללא `rel="noopener noreferrer"`.
- שימוש בטבלה לצורך פריסה.
- שימוש בקישור לפעולה או בכפתור לניווט בלי הצדקה.

טעויות CSS נפוצות

- שכחת סוגר מסולסל.
- שכחת נקודה־פסיק.
- שכחת נקודה לפני שם מחלקה.
- שימוש ב־`#` עבור מחלקה או ב־`.` עבור `id`.
- כפילות של אותם כללים.
- סלקטורים מסובכים מדי.
- הסתמכות על רוחב קבוע בלבד.
- הסרת `outline` בלי חלופה.
- ניגודיות נמוכה.
- תוכן שחורג במסך קטן.

טעויות JavaScript נפוצות

- `querySelector` מחפש `id` שכבר נמחק מה־HTML ומחזיר `null`.
- ניסיון להפעיל פונקציה על אלמנט שלא קיים.
- שינוי HTML בלי לעדכן את JavaScript או CSS.
- שמירת נתונים ישנים שלא תואמים למבנה החדש.

- אי־ניקוי החלטות שתלויות בתוצאה קודמת.
- יצירת כמה מאזינים במקום להשתמש באירוע מרכזי.
- חוסר טיפול בשגיאות `localStorage` או אודיו.

בדיקה לפני הגשה

מבנה

- [ ] יש `DOCTYPE` , `html` , `head` ו־`body` .
- [ ] השפה והכיוון מוגדרים.
- [ ] יש `title` , `description` ו־`viewport` .
- [ ] יש `header` , `nav` , `main` ו־`footer` .
- [ ] היררכיית הכותרות הגיונית.
- [ ] ההזחות עקביות.

תוכן ונגישות

- [ ] לכל תמונה יש `alt` .
- [ ] לכל שדה יש `label` .
- [ ] כל הכפתורים עובדים עם מקלדת.
- [ ] המיקוד נראה.
- [ ] אין הסתמכות על צבע בלבד.
- [ ] הודעות דינמיות נגישות.
- [ ] קישורים מתארים את היעד.

עיצוב

- [ ] אין Inline CSS.
- [ ] מודל הקופסה נשלט.
- [ ] העיצוב עקבי.
- [ ] האתר עובד במחשב ובטלפון.
- [ ] אין גלילה או חיתוך לא מתוכננים.

אינטראקטיביות

- [ ] הטופס נבדק עם ערכים חסרים ותקינים.
- [ ] אפשר לבחור, לשנות ולבטל תוצאה.
- [ ] ההתקדמות מתעדכנת.
- [ ] הזוכה מוצגת אחרי 31 בחירות.
- [ ] דיאלוגים נפתחים ונסגרים.

- [ ] האיפוס עובד בהתאם להגדרה.
- [ ] רענון משחזר את המידע הרצוי.

מדיה ופרסום

- [ ] תמונות נטענות באתר שפורסם.
- [ ] אודיו פועל.
- [ ] סרטון YouTube מוגדר Unlisted או Public.
- [ ] Allow embedding מסומן.
- [ ] הסרטון עובד ב־Netlify.
- [ ] הקישור שנשלח הוא קישור `netlify.app` הציבורי.
- [ ] אין באתר מידע אישי שאין צורך לפרסם.

---

29. דף עזר מהיר

HTML — מה לבחור?

צורך	אלמנט
כותרת ראשית	`h1`
כותרת אזור	`h2`
פסקה	`p`
ניווט	`nav`
אזור נושאי	`section`
תוכן עצמאי	`article`
תוכן משלים	`aside`
רשימה ללא סדר	`ul`
שלבים לפי סדר	`ol`
מונח והגדרה	`dl` , `dt` , `dd`
תמונה והסבר	`figure` , `img` , `figcaption`
נתונים בשורות ועמודות	`table`
קבלת מידע	`form`
פעולה	`button`
מעבר לכתובת	`a`
מצב או תוצאה	`output`
התקדמות	`progress`
חלון ממוקד	`dialog`
שאלה נפתחת	`details` , `summary`

CSS — תזכורת סלקטורים

```
p { }                       /* אלמנט */
.card { }                   /* מחלקה */
#main-content { }           /* מזהה */
.card h3 { }                /* צאצא */
.card > h3 { }              /* ילד ישיר */
button:hover { }            /* מצב */
[required] { }              /* מאפיין */
[aria-pressed="true"] { }  /* מאפיין וערך */
```

CSS — קופסה בסיסית

```
.card {
    width: 100%;
    margin: 1rem 0;
    padding: 1rem;
    background-color: #ffffff;
    border: 1px solid #cbd5e1;
    border-radius: 0.75rem;
}
```

CSS — Flexbox

```
.actions {
    display: flex;
    align-items: center;
    justify-content: center;
    flex-wrap: wrap;
    gap: 1rem;
}
```

CSS — Grid

```
.grid {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 1rem;
}
```

CSS — מסך קטן

```
@media (max-width: 800px) {
    .grid {
        grid-template-columns: 1fr;
    }
}
```

JavaScript — אירוע בסיסי

```
const button = document.querySelector("#start-button");

button.addEventListener("click", () => {
    document.querySelector("#simulator").scrollIntoView();
});
```

---

30. מקורות הקורס שעליהם מבוסס הסיכום

הסיכום נבנה מתוך חומרי הקורס שנמצאו בתיקייה:

- סילבוס הקורס — מבוא לבניית אתרים באינטרנט.
- מצגת מושגי יסוד בבניית אתרים.
- תגיות בסיס ב־HTML5.
- כותרות ועיצוב טקסט ב־HTML5.
- קישורים, שפה ורשימות.
- תמונות, טבלאות ומדיה.
- מצגת טפסים.
- חלק 1 וחלק 2 של CSS3.
- מיקום אלמנטים ב־CSS.
- HTML5 Cheat Sheet.
- CSS3 Cheat Sheet.
- SEO Cheat Sheet.
- Bootstrap.
- טיפים לכתיבת קוד נקי.
- שאלות פתוחות לבחינה.
- הנחיות פרויקט הסיום — אפליקציית SPA.
- קובץ הפרויקט `index.html` של סימולטור מונדיאל 2026.

סיכום אישי

במהלך הקורס עברנו מהבנת הדרך שבה דפדפן ואתר עובדים, דרך בניית מסמך HTML בסיסי, אל מבנה סמנטי מלא, מדיה, טפסים ועיצוב מתקדם יותר באמצעות CSS. למדנו שקוד טוב אינו רק קוד שמציג תוצאה, אלא קוד בעל משמעות, נגיש, רספונסיבי, קריא ומתועד.

פרויקט סימולטור מונדיאל 2026 חיבר את כל הנושאים: מבנה סמנטי, ניווט פנימי, טופס, רשימות, תמונות, וידאו, אודיו, דיאלוגים, עיצוב מצבים, רספונסיביות, SEO, GEO ונגישות. שילוב JavaScript הפך את הדף מאתר מידע סטטי לאפליקציה פעילה ששומרת מצב, מגיבה לבחירות המשתמש ומנהלת תלות בין שלבי טורניר.

הלקח המרכזי הוא שהמחשב עשוי להריץ גם קוד מבולגן, אך אתר מקצועי צריך להיות מובן גם למשתמשים, למפתחים אחרים, למנועי חיפוש, לקוראי מסך ולכלי בינה מלאכותית.

SEO CHEAT SHEET

בניית אתר מקצועית כיום מחייבת לקחת בחשבון במקביל שלושה קהלים שונים:

1. גוגל ומנועי החיפוש המסורתיים
2. בני האדם הגולשים באתר - כולל אנשים עם מוגבלויות/
3. מודלי שפה גדולים המצטטים מידע מהרשת.

החדשות הטובות הן שרוב ההחלטות הנכונות בקוד - מבנה סמנטי, כותרות מסודרות, טקסט תיאורי, תוכן ברור - משרתות את שלושתם בו-זמנית. המסמך הזה מסייע לכתוב קוד שעונה על הצרכים של כל שלושת הקהלים מהיום הראשון.

SEO - Search Engine Optimization - אופטימיזציה למנועי חיפוש. מטרתו להגדיל את הנראות של הדף בתוצאות החיפוש האורגניות של גוגל, לשפר נגישות לכלל המשתמשים, ולהופיע כמקור אמין במודלי שפה גדולים.

נגישות (Accessibility) - עיצוב ובניית אתרים באופן שמאפשר לכל אדם להשתמש בהם, כולל אנשים עם מוגבלויות ראייה, שמיעה, מוטוריקה או קוגניציה. הנגישות מוסדרת בתקן הבינלאומי WCAG ובחקיקה במדינות רבות.

GEO - Generative Engine Optimization - אופטימיזציה למנועי חיפוש מבוססי בינה מלאכותית, כגון ChatGPT, Claude, Gemini ו-Perplexity. בשונה מגוגל שמציג קישורים, מודלי שפה גדולים (LLM) מסנתזים מידע ומצטטים מקורות ישירות בתשובותיהם - מה שהופך את הסמכותיות, הבהירות והמבנה של התוכן לגורמים משמעותיים.

חלק ה- <head>

נושא	הסבר	גוגל	נגישות	LLM / GEO
`<title>`	כותרת הדף המוצגת בכרטיסיית הדפדפן ובתוצאות החיפוש. אורך מומלץ: 50–60 תווים. כדאי שתהיה ייחודית בכל דף	מומלץ מאוד. כדאי לכלול מילת מפתח ראשית	מוקרא על ידי תוכנות הקראה ככותרת הדף	מסייע למודל להבין את נושא הדף ולצטט אותו נכון
`meta description`	תיאור קצר של תוכן הדף, מוצג בתוצאות החיפוש מתחת לכותרת. אורך מומלץ: 150–160 תווים	משפיע על שיעור ההקלקה (CTR) אך לא ישירות על הדירוג	לא רלוונטי	תיאור ברור עשוי להגדיל סיכוי שהמודל ישתמש בדף כמקור
`meta keywords`	מילות מפתח של הדף	מתעלמת על ידי גוגל מאז 2009	לא רלוונטי	לא רלוונטי
`meta viewport`	הגדרת תצוגה רספונסיבית - מבטיח שהדף יוצג נכון במכשירים שונים	מומלץ מאוד. גוגל מעדיף אתרים רספונסיביים (Mobile First)	משפר נגישות במכשירים שונים	לא רלוונטי
`canonical`	תגית המצביעה על הגרסה הראשית של הדף, למניעת תוכן כפול	מסייע למנוע תוכן כפול ומרכז את "משקל" ה-SEO בגרסה אחת	לא רלוונטי	מסייע למודל לזהות את הגרסה הסמכותית של התוכן
`lang`	הגדרת שפת המסמך בתגית `<html>`	מסייע לגוגל להציג את הדף לקהל הנכון	חובה, קובע את שפת ההקראה	מסייע למודל לעבד את השפה בצורה נכונה
`robots`	מורה למנועי חיפוש אם לסרוק ולאנדקס את הדף: `index/noindex`, `follow/nofollow`	מסייע לשלוט בסריקה ואינדוקס של הדף	לא רלוונטי	`noindex` עשוי למנוע גם מסורקי LLM לאנדקס את הדף
Schema Markup	קוד מובנה (JSON-LD) המוסיף לדף מידע מפורש על ישויות: מחבר, תאריך, סוג תוכן, דירוגים וכדומה.	מאפשר תוצאות עשירות: כוכבים, FAQ, breadcrumbs. מסייע להעדיף תוכן רענן	לא רלוונטי	מבנה נתונים שמודלי שפה עשויים לקרוא ישירות. מסייע למודל לדעת אם המידע עדכני
עקביות שמות וכתובות	שמות קבצים וכתובות URL כדאי שיהיו עקביים, תיאוריים ובאותיות קטנות - מומלץ לא לשנות אותם לאחר הפרסום	כתובות URL תיאוריות מסייעות לדירוג ומניעות שגיאות 404	לא רלוונטי	כתובות עקביות מסייעות למודל לזהות את הדף כמקור יציב וסמכותי

מבנה האתר

נושא	הסבר	גוגל	נגישות	LLM / GEO
XML Sitemap	קובץ `sitemap.xml/` המפרט את כל הדפים באתר ומסייע למנועי חיפוש לגלות ולסרוק אותם. חשוב במיוחד לאתרים גדולים	מסייע לגוגל לגלות דפים חדשים ולהבין את מבנה האתר. מומלץ לרשום ב-Google Search Console	לא רלוונטי	מסייע לסורקי LLM למפות את כלל התוכן הזמין באתר
Robots.txt	קובץ `robots.txt/` המגדיר אילו דפים וספריות מותר או אסור למנועי חיפוש לסרוק	מומלץ לכל אתר - מונע סריקה של דפים פנימיים, כפולים או חסרי ערך	לא רלוונטי	סורקי LLM מכבדים את הקובץ - ניתן להגדיר בו חסימה ספציפית
`llms.txt`	קובץ טקסט בשורש האתר המספק למודלי שפה תקציר תמציתי של מטרת האתר, מבנהו ותכניו המרכזיים	לא רלוונטי	לא רלוונטי	עדיין לא סטנדרט רשמי אבל הולך ומתפשט, מסייע למודלים להבין את האתר ולצטט אותו נכון מבלי לסרוק את כלל הדפים
Topic Clusters	ארגון התוכן סביב נושאים מרכזיים: **Pillar Page** - דף ראשי מקיף על נושא רחב, ו-**Supporting Pages** - דפים תומכים המעמיקים בנושאי משנה, כולם מקושרים זה לזה	אחד מאדני ה-SEO המודרני - מחזק את סמכות הנושא (Topical Authority) ועשוי לשפר דירוג	מסייע למשתמשים לנווט בין תכנים קשורים בצורה הגיונית	מודלים נוטים לזהות אתרים עם כיסוי נושאי עמוק כמקורות סמכותיים לצטוט
Crawl Budget	כמות הדפים שגוגל מוכן לסרוק באתר בפרק זמן נתון - רלוונטי בעיקר לאתרים גדולים	כדאי לוודא שגוגל מבזבז את ה-Crawl Budget על הדפים החשובים - מומלץ למחוק דפים ריקים, כפולים או ללא ערך	לא רלוונטי	לא רלוונטי

תוכן הדף

נושא	הסבר	גוגל	נגישות	LLM / GEO
`<h1>`	הכותרת הראשית של הדף - מגדירה את הנושא המרכזי	פעם אחת בלבד. כדאי לכלול מילת מפתח ראשית	מוקרא ראשון על ידי תוכנות הקראה	מסייע למודל להבין מהו הנושא המרכזי של הדף
`<h2>`–`<h3>`	כותרות משנה המחלקות את התוכן לסעיפים ברורים	מסייעות לגוגל להבין מבנה תוכן. כדאי לכלול מילות מפתח	מאפשרות ניווט מהיר לקוראי מסך	מבנה כותרות ברור מסייע למודל לחלץ מידע ולצטט קטעים ספציפיים
מילת מפתח ראשית	המושג המרכזי שעבורו רוצים להופיע בחיפוש - מילה או ביטוי אחד לדף	כדאי לשלב ב- `title`, `h1` ובפסקה `description` הראשונה	לא רלוונטי	כדאי להשתמש בשפה ברורה וטבעית - מודלי שפה מעדיפים תוכן שנכתב לבני אדם
מילות מפתח משניות	מושגים קשורים התומכים במילת המפתח הראשית ומרחיבים את הכיסוי הנושאי	כדאי לפזר באופן טבעי בגוף הטקסט	לא רלוונטי	פורמט שאלות ותשובות (FAQ) מסייע
צפיפות מילות מפתח	שכיחות מילת המפתח ביחס לכלל הטקסט	אין לדחוס - גוגל מזהה ועשוי להוריד דירוג	לא רלוונטי	דחיסת מילות מפתח עשויה לפגוע בהבנת המודל
אורך תוכן	כמות המילים בדף, משפיעה על עומק הכיסוי הנושאי	תוכן מעמיק (1000+) נוטה לדירוג גבוה יותר	תוכן קצר וממוקד עדיף לקוראי מסך	תוכן ארוך ומפורט עשוי להגדיל סיכוי לצטוט, להבנות בכותרות וסעיפים
ייחודיות התוכן	תוכן מקורי שאינו מועתק מאתרים אחרים	גוגל עשוי להעניש תוכן מועתק	לא רלוונטי	העדפת מקורות מקוריים וסמכותיים
רעננות תוכן	עדכון תוכן קיים בתוספת מידע חדש, תאריך עדכון גלוי ושדות `dateModified` ב-Schema	גוגל נוטה להעדיף תוכן עדכני, במיוחד בנושאים המשתנים לעיתים קרובות	לא רלוונטי	מודלים נוטים לצטט מקורות עם תאריך עדכון ברור ועדכני
`<strong>` / `<em>`	הדגשת טקסט חשוב בגוף התוכן	מסייע לגוגל להבין דגשים	תוכנות הקראה מדגישות קטעים אלו	מסייע למודל לזהות נקודות חשובות

E-E-A-T	Experience, Expertise, Authoritativeness, Trustworthiness - מסגרת של גוגל להערכת איכות תוכן. כדאי לכלול: דף מחבר, `author schema`, קישורים לרשתות חברתיות מקצועיות	גוגל נותן משקל לתוכן ממומחים מזוהים	לא רלוונטי	מודלים נוטים לצטט מקורות בעלי סמכות מזוהה: שם כותב, תאריך, מוסד. שילוב סוכן AI בדף מבוסס על תוכן האתר - איכות התוכן הבסיסי משפיעה ישירות על איכות תשובות הסוכן
תוכן אנושי קודם	תוכן שנכתב לבני אדם - ברור, טבעי, ללא דחיסת מילות מפתח מלאכותית	גוגל עשוי להוריד דירוג לתוכן שנכתב למנועי חיפוש בלבד	מסייע למשתמשים עם לקויות קוגניטיביות	מודלי שפה נוטים לתגמל תוכן טבעי - תוכן מלאכותי עשוי להידחות כמקור

דפים מיוחדים

דף	הסבר	גוגל	נגישות	LLM / GEO
דף About	דף המציג את זהות הארגון, הצוות, הערכים והסיפור מאחורי האתר. מומלץ לכלול: שמות כותבים, תפקידים, קישורים לפרופילים מקצועיים ולרשתות חברתיות, ו-Schema מסוג `Person` או `Organization`	מסייע מאוד ל-E-E-A-T - גוגל משתמש בו להעריך את מהימנות האתר	מסייע למשתמשים להבין עם מי הם מתקשרים - חשוב במיוחד לאנשים עם לקויות קוגניטיביות הזקוקים לאמון ובהירות	מודלים עשויים לבדוק דפי About לאימות סמכות המקור. שם כותב, תפקיד ומוסד עשויים להגדיל את הסיכוי לצטוט
דף Q&A / FAQ	דף שאלות ותשובות נפוצות המאורגן בפורמט שאלה–תשובה ברור. מומלץ להוסיף Schema Markup מסוג `FAQPage`	מגדיל סיכוי להופיע ב-Featured Snippets ובתוצאות "People Also Ask"	מאפשר למשתמשים למצוא תשובות מהר ללא קריאת טקסט ארוך	אחד הפורמטים המועדפים על מודלי שפה - מודלים מחפשים תשובות ישירות לשאלות ונוטים לצטט פורמט זה בתדירות גבוהה

מדיה - תמונות, אודיו ווידאו

נושא	הסבר	גוגל	נגישות	LLM / GEO
`alt` לתמונות	טקסט אלטרנטיבי המתאר את תוכן התמונה, מופיע אם התמונה לא נטענת	מסייע לגוגל להבין תוכן התמונה, משפיע על חיפוש תמונות	חובה - מוקרא במקום התמונה לעיוורי מסך	מודלי שפה אינם רואים תמונות - `alt` הוא המידע היחיד שהם מקבלים
שם הקובץ	השם שניתן לקובץ המדיה לפני העלאה לאתר	שמות תיאוריים מסייעים לחיפוש.	לא רלוונטי	לא רלוונטי
גודל קובץ	משקל קובץ המדיה - משפיע ישירות על מהירות טעינת הדף.	קבצים כבדים מאטים טעינה ועשויים להוריד דירוג	קבצים כבדים מקשים על גלישה ברשתות איטיות	לא רלוונטי
`figure>` +> `<<figcaption`	עטיפת תמונה בתגית סמנטית עם כיתוב מוסבר	מסייע לגוגל להבין הקשר התמונה	מספק הקשר נוסף לתוכנות הקראה	הכיתוב מסייע למודל להבין את משמעות התמונה
כתוביות לווידאו (`<track>`)	תגית HTML המוסיפה כתוביות לתוך אלמנט `<video>` בפורמט WebVTT	גוגל עשוי לסרוק את תוכן הכתוביות ולהשתמש בו לדירוג	חובה לפי WCAG: הופכת תוכן וידאו לנגיש לחרשים ולכבדי שמיעה	מודלים אינם שומעים, כתוביות הן המידע היחיד שהם מקבלים
תמלול (Transcript)	טקסט מלא של תוכן הווידאו או האודיו, מופיע בדף כטקסט רגיל	מאפשר לגוגל לסרוק ולאנדקס את כל תוכן הדיבור, מגדיל משמעותית את הנראות	מאפשר לכל משתמש לצרוך את התוכן ללא הפעלת המדיה	מסייע מאוד, מודלים קוראים טקסט בלבד ומצטטים ממנו ישירות
Schema לווידאו (`VideoObject`)	קוד מובנה המוסיף מידע על הווידאו: כותרת, תיאור, תאריך	מאפשר הצגת תוצאות עשירות של וידאו בגוגל	לא רלוונטי	מסייע למודל להבין את הקשר הווידאו ולצטט אותו כמקור

קישורים

נושא	הסבר	גוגל	נגישות	LLM / GEO
קישורים פנימיים	קישורים בין דפים באותו האתר - חלק מרכזי באסטרטגיית Topic Clusters	מסייעים לגוגל לסרוק את האתר ומחזקים דפים חשובים	מאפשרים ניווט ברור באתר	מסייעים למודל להבין את מבנה האתר ואת הקשר בין הדפים
עוגן הקישור (Anchor Text)	הטקסט הנראה והנלחץ על הקישור	טקסט תיאורי הכולל מילת מפתח - עדיף על פני "לחץ כאן"	מומלץ מאוד - "לחץ כאן" חסר משמעות לקוראי מסך	טקסט תיאורי מסייע למודל להבין לאן מוביל הקישור
קישורים חיצוניים	קישורים לאתרים חיצוניים רלוונטיים ואמינים	מסייעים לאמינות הדף - כדאי לקשר למקורות מוכרים	לא רלוונטי	ציטוט מקורות מהימנים עשוי להגביר את הסיכוי שהמודל יראה בדף מקור אמין
`"rel="nofollow`	מאפיין המורה למנועי חיפוש לא לעקוב אחר הקישור ולא להעביר "משקל" SEO	משמש לקישורים ממומנים או לא מהימנים	לא רלוונטי	לא רלוונטי
Backlinks	קישורים נכנסים מאתרים חיצוניים לאתר שלנו	אחד הגורמים המשפיעים ביותר על הדירוג - ככל שהמקשר אמין יותר, כך ההשפעה גדולה יותר	לא רלוונטי	אתר שמצוטט על ידי אתרים אמינים רבים עשוי להיתפס על ידי מודלים כמקור סמכותי

נגישות

נושא	הסבר	גוגל	נגישות	LLM / GEO
ניגודיות צבעים	היחס בין צבע הטקסט לצבע הרקע - משפיע על קריאות לאנשים עם לקות ראייה	לא רלוונטי	חובה לפי WCAG - יחס ניגודיות מינימלי 4.5:1 לטקסט רגיל	לא רלוונטי
`tabindex` וניווט מקלדת	הגדרת סדר ניווט בין אלמנטים אינטראקטיביים באמצעות מקלדת	לא רלוונטי	כל האלמנטים האינטראקטיביים כדאי שיהיו נגישים ממקלדת	לא רלוונטי
`aria-label`	תכונה המוסיפה תיאור טקסטואלי לאלמנטים שאין להם טקסט גלוי, כגון אייקונים וכפתורים	לא רלוונטי	מסייע לתוכנות הקראה לפרש אלמנטים ויזואליים	מסייע למודל להבין תפקיד אלמנטים ויזואליים
סדר לוגי של כותרות	שמירה על היררכיה עקבית: `h1` ← `h2` ← `h3` ← ללא דילוג	מסייע לגוגל להבין מבנה תוכן	חובה - קוראי מסך מנווטים לפי סדר הכותרות	מסייע למודל להפיק מידע בצורה עקבית ומדויקת
`<label>` בטפסים	תגית המקשרת בין שדה קלט לתיאורו הטקסטואלי	לא רלוונטי	חובה - מאפשר לתוכנות הקראה לפרש שדות טופס	לא רלוונטי
שפה פשוטה וברורה	כתיבה בשפה נגישה, ללא מינוח מיותר, עם משפטים קצרים וממוקדים	תוכן קריא עשוי להוריד Bounce Rate ולשפר דירוג	מסייע למשתמשים עם לקויות קוגניטיביות	מודלים נוטים לחלץ ולצטט תוכן ברור וחד-משמעי בקלות רבה יותר
מבנה סמנטי	שימוש נכון בתגיות HTML5 סמנטיות: `header`, `main`, `article`, `section` במקום `div` גנרי	מסייע לגוגל להבין את מבנה הדף ותפקיד כל חלק	תוכנות הקראה מסתמכות על תגיות סמנטיות לניווט	מודלים נוטים להבין טוב יותר תוכן המחולק לאזורים סמנטיים ברורים

HTML5 CHEAT SHEET

תגיות יסוד		
תגית פותחת	הסבר	**תגית סוגרת**
<!DOCTYPE html>	**חובה**. יכול להופיע **פעם אחת** במסמך. מגדיר את סוג הקוד עבור דפדפן, כ- HTML5	
<!--XXX-->	**הוספת הערה. רשות**. יכול להופיע **כמה פעמים שרוצים** במסמך. מגדיר את מה שכתוב בין הסימנים כהערה, שהדפדפן אינו קורא. בתעשייה עדיף לכתוב הערות באנגלית. בקורס נכתוב בעברית כדי לוודא הבנה.	
<html>	**חובה**. יכול להופיע **פעם אחת** במסמך. מופיע מיד אחרי סוג המסמך (Doctype), והתגית הסוגרת בסוף המסמך.	</html>
<head>	**חובה**. יכול להופיע **פעם אחת** במסמך. כולל בתוכו את הגדרות המסמך. חובה לכלול בתוכו את התגית title. שאר התגיות רשות. **לא להתבלבל עם חלק ה head, התגית header או כותרות h1 עד h6.**	</head>
תגיות מטה-דאטא בתוך חלק ה head		
<title>	**חובה**, בתוך חלק ה- head. יכולה להופיע **פעם אחת** במסמך. התוכן מוצג בכותרת הדף בדפדפן, ובתוצאות החיפוש בגוגל. **לא להתבלבל עם חלק ה head, התגית header או כותרות h1 עד h6.**	</title>
<meta>	בתוך חלק ה- head**.** כל אלמנטי המטה, צריכים להופיע ברצף. חייבים להופיע: charset, description, keywords, author	
<style>	**רשות**, בתוך חלק ה head. **לא מומלץ להשתמש בה.** עדיף לשלב עיצוב בקובץ CSS3 חיצוני.	</style>
<link>	**רשות**, בתוך חלק ה- head. כל אלמנטי ב- link צריכים להופיע ברצף. משמש כקישור לקובץ CSS3 לעיצוב התוכן, קבצי פונט וכדומה.	
<body>	**חובה**. יכול להופיע רק פעם אחת במסמך. כולל בתוכה את כל התוכן של המסמך.	</body>
תגיות מבנה סמנטיות		
<header>	מגדיר את החלק העליון של הדף. יכול להכיל כל תגית למעט header, main, aside ו- footer. **לא להתבלבל עם חלק ה head , ה- title , או כותרות h1 עד h6.**	</header>
<nav>	מאפשר ליצור תפריט ניווט. ניתן לכלול במסמך מספר פעמים. אפשר וכדאי לקנן בתוכו רשימה.	</nav>
<main>	מגדירה את האזור המרכזי בדף. יכול להופיע רק פעם אחת במסמך. יכול להכיל כל תגית למעט header, footer, aside, nav	</main>
<aside>	תמיד צריך לכלול בתוכו כותרת (h2 - h6). מגדיר אזור צדדי בדף, למשל חלק של חדשות. ניתן לכלול במסמך מספר פעמים.	</aside>

	לא משפיע על העיצוב, יוצג בצד רק אם נעצב זאת.	
<section>	תמיד צריך לכלול בתוכו כותרת (h2 - h6). מגדיר מקטע / אזור מסוים במסמך, חלק מה- main. ניתן לכלול במסמך מספר פעמים. מצמצם את הצורך ב div.	</section>
<article>	תמיד צריך לכלול בתוכו כותרת (h2 - h6). מגדיר אזור עצמאי מסוים במסמך, העומד בפני עצמו. ניתן לכלול במסמך מספר פעמים.	</article>
<footer>	מגדיר את החלק התחתון של הדף. יכול להכיל כל תגית למעט header, main ,aside ו- footer.	</footer>
אלמנט המתפקד כ"מיכל" (container) **לצרכי עיצוב, ללא משמעות סמנטית**		
	מאפשר לאגד מספר אלמנטים מסוג בלוק ביחד לצורך עיצוב ב CSS. ללא משמעות סמנטית. יש להעדיף תגיות מבנה סמנטיות, ולהשתמש ב div רק לצורך עיצוב.	
	מאפשר לאגד מספר אלמנטים מסוג in-line ביחד לצורך עיצוב ב CSS. ללא משמעות סמנטית.	
תגיות להגדרת כותרת * לא להתבלבל עם חלק ה head של המסמך, עם ה- title או עם התגית header.		
<h1>	הכותרת הכי חשובה בתוכן הדף. חובה להשתמש , ורק פעם אחת. התוכן מקבל חשיבות רבה במנוע החיפוש גוגל.	</h1>
<h2>	כותרת עם חשיבות משנית, אפשר להשתמש כמה פעמים במסמך. התוכן ממופה על ידי מנוע החיפוש גוגל.	</h2>
<h3>	כותרת עם חשיבות משנית, אפשר להשתמש כמה פעמים במסמך. התוכן ממופה על ידי מנוע החיפוש גוגל.	</h3>
<h4>	כותרת עם חשיבות זניחה עבור מנוע החיפוש, אפשר להשתמש כמה פעמים במסמך.	</h4>
<h5>	כותרת עם חשיבות זניחה עבור מנוע החיפוש, אפשר להשתמש כמה פעמים במסמך.	</h5>
<h6>	הכותרת הכי פחות חשובה בדף, אפשר להשתמש כמה פעמים במסמך.	</h6>
תגיות לעיצוב טקסט * ניתן לקנן את התגיות אחת בתוך השניה עדיף לא להשתמש בהן, אלא לשלב בקובץ CSS3		
	קיצור של **bold**. הטקסט המודגש כתוב בין התגית הפותחת לסוגרת.	
	קיצור של *Italics*. הטקסט המוטה כתוב בין התגית הפותחת לסוגרת.	
<mark>	מדגיש את הטקסט המופיע בין התגית הפותחת לסוגרת עם מדגש (מרקר).	</mark>

	ברירת מחדל לעיצוב - צהוב. אבל ניתן להגדיר צבעים אחרים באמצעות קוד CSS3.	
^{	הופך את הטקסט הכתוב בין התגית הפותחת לסוגרת לכתב עילי.	}
_{	הופך את הטקסט הכתוב בין התגית הפותחת לסוגרת לכתב תחתי.	}
<small>	הופך את הטקסט הכתוב בין התגית הפותחת לסוגרת לקטן יותר משאר הטקסט. * יש לשים לב שאין יותר אלמנט big	</small>
 	מעבר לשורה חדשה. אין תגית סוגרת.	
<hr>	קיצור של horizontal ruler, תגית שמוסיפה קו אופקי.	
	עיצוב סמנטי של טקסט * מסייע להבנת המהות עבור תוכנות ההקראה ומנועי החיפוש	
	מציין שהטקסט בין התגית הפותחת לסוגרת הוא טקסט חשוב - חזק - strong. ברירת מחדל לעיצוב - bold.	
	מציין שהטקסט בין התגית הפותחת לסוגרת הוא טקסט חשוב-מודגש-emphasized. ברירת מחדל לעיצוב - italic.	
<dfn>	מציין להסבר מושג. * לא להתבלבל עם רשימת מושגים	</dfn>
<del>	הצגת טקסט מבוטל, עם קו אופקי לאורך הטקסט (בעבר היה strike שלא פעיל יותר).	</del>
<ins>	קיצור של insert, הוספה של טקסט חדש, פעמים רבות מיד אחרי del.	</ins>
<time>	מתרגם את הטקסט בין התגית הפותחת לתגית הסוגרת לפורמט זמן. מסייע להבנת הקשר, תוכנות הקראה והעברת מידע ללוחות שנה.	</time>
<address>	מצביע על מידע מסוג כתובת. משמש להכנסת פרטי קשר. ברירת המחדל מוצג מוטה - italic.	</address>
<figure>	משלב תמונה בתוך אלמנט, עם כותרת תחתית לתמונה: <figcaption>	</figure>
<cite>	משמש לאיזכור שם של יצירה, ספר, תוכנית, אלבום וכדומה. הטקסט מופיע ב-italic.	</cite>
<q>	ציטוט קצר.הטקסט מוצג בתוך מרכאות. כדאי להוסיף ציטוט באופן הזה כדי שגוגל לא "תחשוד" שעברתם על זכויות יוצרים.	</q>
<blockquote>	ציטוט ארוך. הטקסט מוצג בתוך מרכאות. כדאי להוסיף ציטוט באופן הזה כדי שגוגל לא "תחשוד" שעברתם על זכויות יוצרים.	</blockquote>
<details>	מאפשר להסתיר פרטים בטקסט. התוכן שבין התגית הפותחת לסוגרת מכווץ - ניתן לפתוח על ידי הקשה על החץ הקטן. את הטקסט שרוצים שיופיע בדף, משלבים בתוך <summary> את הטקסט שמגלים, משלבים בתוך פסקה רגילה.	</details>
<abbr>	הוספת ראשי תיבות.	</abbr>

רשימות		
	קיצור של Unordered List, בפתיחה ובסיום של רשימה לא ממוספרת.	
	קיצור של Ordered List, בפתיחה ובסיום רשימה ממוספרת.	
	מסמן פריט ברשימה, ממוספרת או לא. ניתן לבחור מאפיין לעיצוב הפריטים.	
<dl>	קיצור של Definition List, בפתיחה ובסיום של רשימת הגדרות.	</dl>
<dt>	קיצור של Definition Term, תגית ההגדרה.	</dt>
<dd>	קיצור של Definition Description, תגית ההסבר.	</dd>
טבלאות *לעצב רק דרך ה CSS		
<table>	**חובה**. תגית המכריזה על פתיחת טבלה, ולאחריה מגיעות התגיות הפנימיות.	</table>
<caption>	**רשות**. הגדרת כותרת חיצונית לטבלה, תמיד מתחת לתגית table .	</caption>
<tr>	**חובה**. קיצור של table row, הגדרת שורה בטבלה.	</tr>
<td>	**חובה**. קיצור של table data, הגדרה של תוכן התא.	</td>
<th>	**רשות**. הגדרה של שורת כותרת, במקום td	</th>
<thead>	**רשות**. הגדרה של חלק הכותרת לטבלה, בתוכו משלבים את ה tr	</thead>
<tbody>	**רשות**. הגדרה של גוף הטבלה, לשימוש רק אם מגדירים שורת כותרת ו/או שורת סיום.	</tbody>
<tfoot>	**רשות**. הגדרה של שורות סיכום לטבלה.	</tfoot>
הוספת תוכן חיצוני *עדיף לעצב ב- CSS		
<a>	יצירת קישור בין דפים. חובה לצרף את המאפיין href. אפשר לצרף את המאפיין target. בשימוש עם target="_blank" — חובה להוסיף rel="noopener noreferrer" למניעת פרצת אבטחה שמאפשרת לדף החדש לגשת לדף המקורי. אפשר להפנות לדואר ברירת מחדל עם המאפיין mailto.	
<img>	מומלץ לשלב בתוך figure , יחד עם figcaption. הצגת תמונה מתוך קובץ מצורף. חובה לציין את המאפיין src. חובה להוסיף את המאפיין alt, טקסט אלטרנטיבי שיוצג אם התמונה לא תעלה. המאפיין title, מוסיף טקסט הסבר קצר שמופיע כ-tooltip בריחוף העכבר מעל התמונה - כדאי להשתמש בו כשרוצים להוסיף מידע נוסף שאינו חיוני, כמו שם הצלם או הקשר לתמונה — אך לא כתחליף ל-alt. *לא להתבלבל עם ה - title שב- head	
<video>	הטמעה של סרטון וידאו מתוך המחשב או מהרשת.	</video>

<audio>	הטמעה של קובץ אודיו מתוך המחשב או מהרשת.	**</audio>**
<iframe>	הטמעה של פריט HTML אחר (קובץ, וידאו, אודיו וכדומה) בתוך דף האינטרנט.	**</iframe>**
<embed>	הטמעה של פריט חיצוני, פלאג או פאליקציה, בתוך דף האינטרנט.	
טפסים		
<form>	הוספת טופס בתוך דף HTML5.	**</form>**
<input>	**חובה.** מגדיר את סוג הקלט. יש מגוון רחב מאוד של **type** (טקסט, תאריך, מספר, צבע...). **אין תגית סוגרת.**	
<label>	מופיע תמיד לפני כל input. **לא חובה, אבל מומלץ מאוד**. מסביר לגולש את סוג הקלט הנדרש ב input.	**</label>**
<button>	שילוב "כפתורים" בתוך הטופס. הקלקה על הכפתור מבצעת פעולה (הודעה, קישור וכו')	**</button>**
<textarea>	פותח חלון שבו הגולש יכול לכתוב טקסט באופן חופשי. ניתן לשלוט בגודל החלון.	**</textarea>**
<select>	מאפשר לבחור פריט אחד מתוך רשימה נפתחת.	**</select>**
<fieldset>	מרכז מספר אלמנטים בטופס ביחד (בתוך טופס לא משתמשים ב sectin או article וכדומה).	**</fieldset>**
<legend>	מוסיף כותרת ל fieldset, עבור קבוצת אלמנטים בטופס (בתוך טופס לא משתמשים ב h1 - h6 או פ וכדומה).	**</legend>**

מספר כללים חשובים לכתיבת קוד:

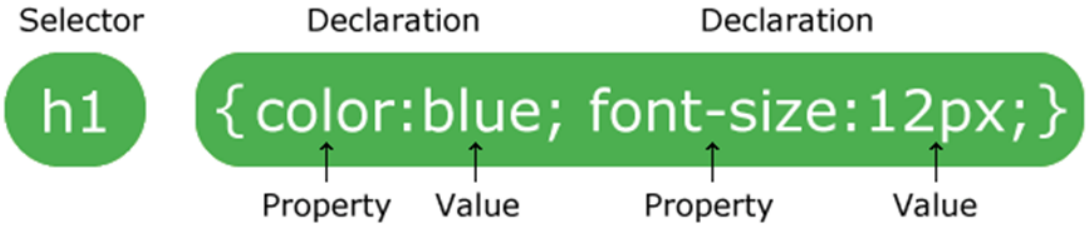
- ☐ המסמך יתחלק ל 2 חלקים - head ו- body.
- ☐ **בתגית** html יש להוסיף את מאפיין השפה <html lang="he">
- ☐ שלבו הערות באנגלית, המסבירות מה אתם מבצעים לאורך כל הקוד
- ☐ עבור שמות תיקיות, קבצים, אלמנטים, תגיות ומאפיינים – השתמשו באותיות אנגלית קטנות, ללא רווחים
- ☐ הקפידו לסגור כל אלמנט עם תגית פותחת וסוגרת, אין צורך לסגור תגיות ריקות
- ☐ שימו את הערכים של המאפיין (attributes) תמיד במרכאות
- ☐ הימנעו משורות קוד ארוכות
- ☐ הימנעו משורות קוד ריקות
- ☐ קינון התגיות: תגיות יש להן תגית סוגרת - יסגרו בהתאמה לסדר פתיחתן (LIFO - Last In First Out)
- ☐ אין לשנות שמות של קבצים וספריות תוך כדי הכתיבה!
- ☐ אחרי כל תו מיוחד, כמו   חובה לשים ; נקודה פסיק

**** אלמנטי META מומלצים:**

סט התווים הנדרש להצגה תקינה של התוכן כולל עברית ואימוג'י)	<meta charset="utf-8">

תיאור נושא הדף	<meta name="description" content="כאן מכניסים את תיאור הדף בשפה המתאימה">
הוספת מילות מפתח	<meta name="keywords" content="כאן מכניסים את מילות הקוד, בשפה המתאימה, עם פסיק בין מושג למושג">
ציון שם כותב התוכן	<meta name="author" content="כאן מכניסים ת שם כותב התוכן">
הפיכת האתר לרספונסיבי	<meta name="viewport" content="width=device-width, initial-scale=1.0">

CSS3 CHEAT SHEET



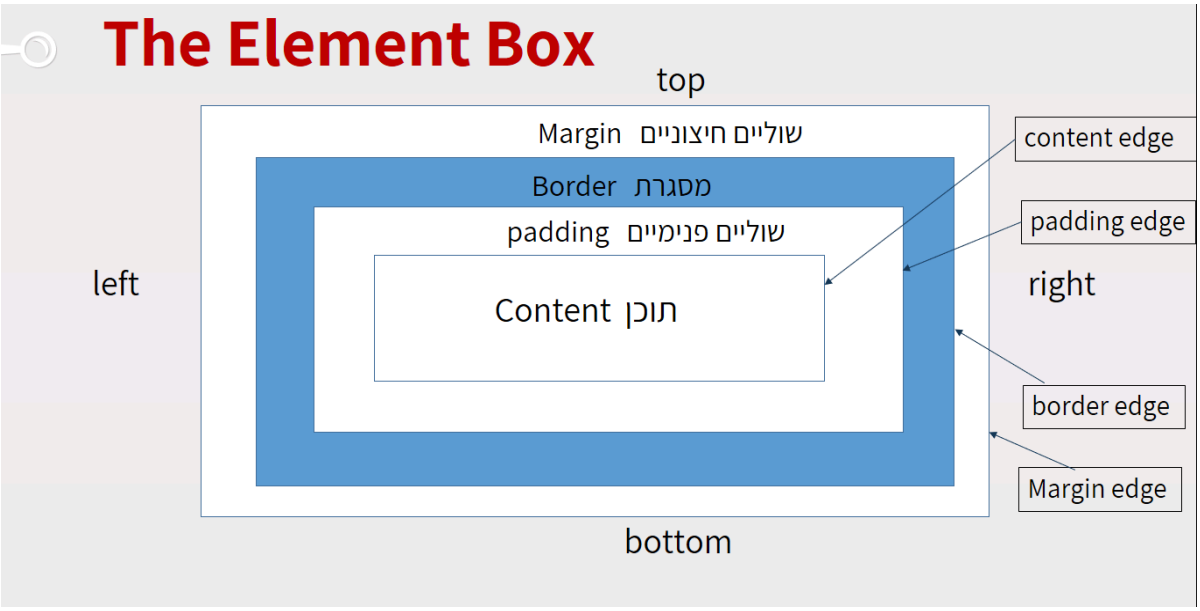
כאשר יש כמה הכרזות לסלקטור ספציפי, מקובל למיין אותם על פי סדר עולה של ה ABC.
אני ממליצה למיין לפי ההגיון, ולהקפיד על אותו הסדר לאורך כל הקובץ.
כל הכרזה (declaration) צריכה להופיע בשורה נפרדת.
ה - selector יכול להתייחס לשם של אלמנט / אלמנטים / * (כל האלמנטים) , ל id או ל class.

רקע	צבע רקע*	מאפשר להגדיר צבע רקע לדף כולו או לאלמנט מסוים	Background-color
טקסט	צבע טקסט*	מאפשר להגדיר צבע לטקסט	color
	יישור טקסט	יישור הטקסט בתוך האלמנט: left, right, center, justify	text-align
	קישוט הטקסט	הוספת קישוט לטקסט: none, underline, overline, line-through	text-decoration
	גובה הטקסט	הגדרת גובה השורה, משפיע על קריאות הטקסט: מספר, פיקסלים (px), אחוזים (%) או normal	line-height
	מרווח האותיות	הגדרת הרווח בין האותיות: פיקסלים (px) — ערך חיובי להרחבה, שלילי לצמצום	letter-spacing
	סגנון הפונט	normal, **italic,** oblique	font-style
	אותיות גדולות קטנות	כאשר כותבים בשפה האנגלית	font-variant
	עובי הפונט	**bold**, bolder, lighter, normal, inherit, 100-900	font-weight
	גודל פונט	xx-small, x-small, small, medium, large, x-large, xx-large או בפיקסלים (px), או ביחסיות (em) או באחוזים	font-size
	משפחת הפונטים	רשימת הפונטים הבטוחים לשימוש, ואופן ההכרזה עליהם	font-family
	הגדרה מקוצרת לפונט	מאפשר בהכרזה אחת להגדיר את 5 הערכים של עיצוב הפונט	font
רשימה	סגנון עיצוב רשימה	בחירת הסגנון מתוך מגוון מוצע, כולל אפשרות לעברית	list-style-type
	מיקום התבליטים	בתוך או מחוץ למסגרת	list-style-position
	הצגת image	קישור לקובץ מתאים	list-style-image

		כתבליט	
list-style	מאפשר בהכרזה אחת להגדיר את 3 מאפייני עיצוב הרשימה	הגדרה מקוצרת לעיצוב רשימה	
padding-top	הגדרת שוליים פנימיים (לתוכן) - לחלק העליון	padding	מודל הקופסא שוליים פנימיים
padding-right	הגדרת שוליים פנימיים (לתוכן) - לחלק הימני		
padding-bottom	הגדרת שוליים פנימיים (לתוכן) - לחלק התחתון		
padding-left	הגדרת שוליים פנימיים (לתוכן) - לחלק השמאלי		
padding Top Right Bottom Left	הגדרה מקוצרת של 4 צדדים (למעלה, ימינה, למטה, שמאלה)		
padding T RL B	הגדרה מקוצרת של 3 צדדים (למעלה, ימינה ושמאלה, למטה)		
padding TB RL	הגדרה מקוצרת של 2 צדדים (למעלה ולמטה, ימינה ושמאלה)		
padding TRBL	הגדרה מקוצרת של 4 הצדדים באותו הרוחב		
border-width	הגדרה של רוחב גבולות לאלמנטים	רוחב גבול	מודל הקופסא - גבולות
border-style	הגדרה של סגנונות שונים של גבולות לאלמנטים	סגנון גבול	
border-color	הגדרה של צבע גבולות לאלמנטים	צבע גבול	
border	הגדרה מקוצרת ל 3 הערכים (רוחב, סגנון, צבע) של עיצוב הגבול	הגדרה מקוצרת לגבול	
border-top	הגדרה מקוצרת ל 3 הערכים של עיצוב הגבול העליון בלבד		
border-right	הגדרה מקוצרת ל 3 הערכים של עיצוב הגבול הימני בלבד		
border-bottom	הגדרה מקוצרת ל 3 הערכים של עיצוב הגבול התחתון בלבד		
border-left	הגדרה מקוצרת ל 3 הערכים של עיצוב הגבול השמאלי בלבד		
margin-top	הגדרת שוליים חיצוניים - לחלק העליון	margin	מודל הקופסא שוליים חיצוניים
margin-right	הגדרת שוליים חיצוניים - לחלק הימני		
margin-bottom	הגדרת שוליים חיצוניים - לחלק התחתון		
margin-left	הגדרת שוליים חיצוניים - לחלק השמאלי		
margin T R B L	הגדרה מקוצרת של 4 צדדים (למעלה, ימינה, למטה, שמאלה)		
margin T RL B	הגדרה מקוצרת של 3 צדדים (למעלה, ימינה ושמאלה, למטה)		
margin TB RL	הגדרה מקוצרת של 2 צדדים (למעלה ולמטה, ימינה ושמאלה)		
margin TRBL	הגדרה מקוצרת של 4 הצדדים באותו הרוחב		

מודל הקופסא גודל כולל	חישוב גודל הקופסא	קובע כיצד מחושב הגודל הכולל של הקופסא — האם ה- padding וה- border נכללים בתוך ה- width וה- height או מתווספים מעליהם. מומלץ להגדיר border-box על כל האלמנטים כדי למנוע חישובים לא צפויים של גדלים: content-box (ברירת מחדל — padding ו- border מתווספים לרוחב) border-box (מומלץ — padding ו- border כלולים בתוך הרוחב)	box-sizing

*קישור לטבלת שמות וקודים של הצבעים



פונטים:

פונטים מובנים, בטוחים לשימוש - https://www.w3schools.com/cssref/css_websafe_fonts.php

הבדלים בין משפחות פונטים - https://www.w3schools.com/css/css_font.asp

הטמעת פונטים חיצוניים של גוגל באמצעות קישור - https://www.w3schools.com/howto/howto_google_fonts.asp

קישור לפונטים של גוגל - https://fonts.google.com/

אייקונים:

אייקונים של Bootstramp -

https://icons.getbootstrap.com/

הוספת link ב head של הקובץ:

```
<link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css">
```

תרגיל סיום: בניית אפליקציה אינטראקטיבית כ־Single Page Application

התרגיל הוא אישי, ומקנה 20% מהציון הסופי. יש להעלות את התרגיל ללמדה עד ה- 30.6

בתרגיל זה תבנו אפליקציה אינטראקטיבית בעמוד אחד, באמצעות HTML, CSS ו־JavaScript בלבד.
האפליקציה יכולה להיות יחידת לימוד אינטראקטיבית למשל עם שאלון רב־ברירה, משחק פשוט למשל משחק זיכרון, רביעיות או טטריס, סימולציה פשוטה או כלי תרגול. התוכן כולו נתון להחלטתכם.
אין להשתמש ב־React, Vue, Angular או כל Framework אחר.
* שימו לב! מבחינה טכנית אנחנו בונים אתר, שמבחינת ההתנהגות, היא מדמה אפליקציה.

מטרת התרגיל

מטרת התרגיל היא לבנות אפליקציה אינטראקטיבית, רספונסיבית, נגישה ומעוצבת, תוך הקפדה על HTML סמנטי, CSS מסודר ונקי, מבנה עמוד תקין, נגישות, SEO בסיסי, ארגון JavaScript ברור והתאמה לקריאה גם על ידי כלי בינה מלאכותית.

שימו לב!

המטרה אינה רק ליצור אפליקציה שעובדת.
המטרה היא להגיש אפליקציה שלמה, מסודרת, סמנטית, נגישה, מעוצבת וברורה, שמראה שליטה אמיתית בעקרונות שנלמדו בקורס.

קובץ ההגשה

קובץ אחד בלבד: index.html, שיכלול את כל חלקי האפליקציה: מבנה ותוכן, עיצוב ולוגיקה אינטראקטיבית.

מבנה האפליקציה

האפליקציה צריכה לכלול מסך פתיחה, רכיב אינטראקטיבי מרכזי, הוראות שימוש, אזור About על יוצר/ת האפליקציה, תפריט ניווט פנימי, מדיה הכוללת תמונה, וידאו ואודיו, רשימות, קישורים, טבלה ותפריטים.
מומלץ לשלב טפסים, אבל אני לא מחייבת.
האפליקציה חייבת לכלול סרטון, מקסימום 5 דקות, שבו אתם מסבירים על האפליקציה בדגש על הקוד עצמו. ניתן להעלות את הסרטון ליוטיוב ולשתף משם, זה יכול להתאים בדף ה- About. מטרת הסרטון להוכיח שאתם מבינים את הקוד כולו.
אפשר ליצור כמה אזורים, טאבים או קישורים פנימיים לפי הצורך, כל עוד הכול נשאר בתוך אותו קובץ.

HTML סמנטי

יש להשתמש במבנה סמנטי תקין ובתגיות מתאימות לתוכן, כולל header, nav, main, section, article, aside, footer.
יש להקפיד על היררכיית כותרות תקינה, מבנה ברור, קישורים תקינים ותוכן קריא ומאורגן.
בשונה מהמקובל בתעשייה שם נעשה שימוש רחב בתגיות div ו- span, בתרגיל זה אין להשתמש בהן כלל. מטרת ההגבלה לצורך תרגיל זה היא לעודד שימוש נכון בתגיות סמנטיות ובמבנה HTML משמעותי.

CSS

בשונה מהמקובל בתעשייה, שאז מקפידים על קבצי CSS חיצוניים, בתרגיל זה, כל ה־ CSS יופיע בתוך אלמנט style באזור ה־head.
העיצוב צריך להיות מודרני, נעים לעין, עקבי, רספונסיבי, נגיש ומותאם לנושא האפליקציה.
יש להימנע מעיצוב מבולגן, כפילויות מיותרות, קוד לא מאורגן ושימוש ב־inline styling.

JavaScript

האפליקציה חייבת להיות אינטראקטיבית ולכן יש להשתמש ב־JavaScript.

מכיוון שלא למדנו JavaScript , זוהי הזדמנות להתנסות ב -Vibe Coding, אבל יש להקפיד במיוחד על סדר וארגון.
כל קוד ה־JavaScript צריך להיות מרוכז במקום אחד, בסוף ה־ body, ולא מפוזר בתוך תגיות HTML.
יש לחלק את הקוד לאזורים ברורים באמצעות הערות, למשל: משתנים כלליים, בחירת אלמנטים, פונקציות, לוגיקה אינטראקטיבית, אירועים והפעלת האפליקציה.

תיעוד הקוד

יש לתעד את הקוד באמצעות הערות ברורות וקצרות לאורך הקובץ. ההערות צריכות להסביר את מבנה האפליקציה, אזורים מרכזיים בקוד, חלקי JavaScript חשובים, אלמנטים מיוחדים או החלטות משמעותיות בתהליך הפיתוח. מטרת התיעוד היא להפוך את הקוד לקריא, מאורגן וקל להבנה גם עבור מי שלא כתב אותו.
ההערה הראשונה בקובץ צריכה לכלול את השם המלא של כותב/ת הקוד ואת תאריך כתיבת העבודה.
למרות שבתעשיית התוכנה מקובל בדרך כלל לכתוב הערות באנגלית, לצורך תרגיל זה ההערות ייכתבו בעברית, כדי לשמור על אחידות ובהירות במסגרת הקורס.

ספריות חיצוניות

בתרגיל זה מותר להשתמש בקישורי CDN (Content Delivery Network), כלומר קישורים חיצוניים לקבצים הנמצאים בשרתים באינטרנט ונטענים ישירות לתוך העמוד, ללא צורך להוריד או להעלות את הקבצים ידנית לפרויקט. שימוש כזה מאפשר לעבוד בצורה פשוטה, מסודרת ויעילה עם פונטים, אייקונים ואנימציות בסיסיות.
בתרגיל זה מותר להשתמש ב־CDN עבור ספריות כמו Google Fonts, Font Awesome, Bootstrap Icons או Animate.css בלבד.
אין להשתמש ב־CDN עבור Frameworks או ספריות שמחליפות את כתיבת ה־HTML, CSS או JavaScript הבסיסיים של התרגיל, כגון React, Vue, Angular, Tailwind, jQuery, Phaser, GSAP או Three.js.

שימוש בכלי בינה מלאכותית

מותר להשתמש בכלי AI לצורך יצירת קוד, תכנון, עיצוב, debugging ופתרון בעיות.
אפשר להשתמש בכל כלי שתרצו.
עם זאת, האחריות על הקוד היא שלכם בלבד.
עליכם לעבור על כל שורת קוד, להבין מה הקוד עושה, לנקות כפילויות, להסיר קוד מיותר ולוודא שהקוד עומד בהנחיות הקורס.
קוד מסורבל, לא קריא, לא סמנטי או לא תקין יפגע בציון גם אם "הכול עובד".

מדיה

אפשר להשתמש במדיה חיצונית באמצעות קישורים.
אם משתמשים בקבצים מ־Google Drive או מכל שירות שיתוף אחר, חובה לוודא שהקבצים פתוחים לצפייה לכל מי שיש לו קישור.
מדיה שלא נטענת בזמן הבדיקה תיחשב כאילו אינה קיימת.

SEO ו־GEO

באפליקציה מסוג Single Page Application קיימים אתגרי SEO ו־GEO ייחודיים, משום שרוב התוכן והאינטראקטיביות נטענים באופן דינמי באמצעות JavaScript בתוך עמוד יחיד. בניגוד לאתר מרובה עמודים, שבו לכל עמוד יש מבנה ותוכן נפרדים, כאן מנועי חיפוש וכלי בינה מלאכותית נדרשים להבין מסמך אחד מורכב ודינמי. לכן יש חשיבות גבוהה במיוחד למבנה HTML סמנטי, להיררכיית כותרות תקינה, לתוכן ברור ומאורגן ולטקסטים שקל למנועי חיפוש ולמודלי AI לנתח ולהבין.

יש להקפיד על כותרת עמוד ברורה, תיאור עמוד מתאים, כותרות מסודרות, טקסט חלופי לתמונות, תיאור ברור של מטרת האפליקציה, תוכן ברור באזור About (הכולל: הסבר קצר על האפליקציה, פירוט באילו כלי AI השתמשתם, תיאור קצר של שינויים או תיקונים שביצעתם בקוד, סרטון שמסביר את הקוד, ורפלקציה קצרה על התרגיל), ואזור שאלות ותשובות (Q&A).

שילוב אלמנטים חדשים

יש לשלב בקוד לפחות שני אלמנטים של HTML שלא נלמדו בכיתה.

לפני כל אלמנט חדש יש להוסיף הערה בקוד שמסבירה מהו האלמנט, מה התפקיד שלו, ומה הסיבה לשילוב שלו דווקא במקום זה באפליקציה.

מטרת הדרישה היא לעודד חקירה עצמאית, הרחבת הידע מעבר לחומר שנלמד בכיתה, ושימוש מושכל בתגיות HTML בהתאם למשמעות התוכן ולא רק לפי מראה עיצובי.

נגישות

יש להקפיד על טקסט קריא, ניגודיות צבעים טובה, ניווט ברור, כפתורים וקישורים ברורים, שימוש נכון ב־alt ל תמונות ומבנה כותרות תקין.

פרסום והגשה

בסיום העבודה יש להעלות את קובץ index.html ל־Netlify.

את הקישור שקיבלתם מ – Netlify יש להעלות ללמדה.

השוואה בין אתר/אפליקציה אינטראקטיביים לבין אתר/אפליקציה סטטיים

מס׳	היבט להשוואה	אפליקציה / אתר אינטראקטיביים	אפליקציה / אתר לא אינטראקטיביים / סטטיים
1	הגדרה	מאפשרים למשתמש לבצע פעולות ולקבל תגובה מהמערכת	מציגים מידע בעיקר לקריאה או צפייה, ללא תגובה משמעותית לפעולות המשתמש
2	תפקיד המשתמש	המשתמש פעיל: לוחץ, בוחר, מזין מידע, משחק, עונה או משנה תצוגה	המשתמש בעיקר צופה או קורא את התוכן
3	שימוש ב־JavaScript	בדרך כלל נדרש JavaScript כדי ליצור תגובות, בדיקות, חישובים ועדכונים במסך	ייתכן שאין JavaScript כלל, או שיש שימוש מינימלי בלבד
4	דוגמאות	משחק זיכרון, שאלון רב־ברירה, מחשבון, סימולציה, טופס עם משוב מיידי	דף מידע, מאמר, עמוד אודות, גלריה פשוטה, דף הוראות
5	חוויית משתמש	דינמית, פעילה, מערבת ולעיתים מותאמת אישית	פשוטה, יציבה וצפויה, אך פחות מערבת
6	למידה ותרגול	מתאימים במיוחד ללמידה פעילה, תרגול, בדיקת ידע ומשוב מיידי	מתאימים להצגת הסברים, מידע, סיכומים ותוכן לקריאה
7	מורכבות פיתוח	מורכבים יותר: צריך לתכנן לוגיקה, אירועים, מצבי מערכת ובדיקות	פשוטים יותר: מתמקדים במבנה תוכן, עיצוב ונגישות

8	נגישות	דורשים תשומת לב מיוחדת לכפתורים, מקלדת, הודעות מצב ו־`aria-live`	נגישות מתמקדת בעיקר בכותרות, קישורים, טקסט חלופי וניגודיות
9	ביצועים	עלולים להיות כבדים יותר אם יש הרבה קוד, אנימציות או חישובים	לרוב קלים ומהירים יותר לטעינה

השוואה בין Single Page Application לבין Multi-Page Application

מס׳	היבט להשוואה	אפליקציה אינטראקטיבית ב־SPA — Single Page Application	אפליקציה אינטראקטיבית ב־MPA — Multi-Page Application
1	מבנה	כל האפליקציה נמצאת בעמוד HTML אחד מרכזי	האפליקציה מחולקת למספר עמודים נפרדים
2	מעבר בין אזורים	מעבר בין אזורים באותו עמוד, למשל פתיחה, משחק, הוראות ו־About	מעבר בין דפים שונים, למשל home.html, game.html, about.html
3	טעינת עמודים	העמוד נטען פעם אחת, וה־JavaScript משנה את התוכן או המצב במסך	בכל מעבר לדף אחר נטען עמוד חדש מהשרת או מהתיקייה
4	האינטראקציה	מתרחשת בתוך אותו עמוד: משחק, שאלון, טופס או סימולציה	יכולה להתרחש בכל עמוד בנפרד, למשל משחק בדף אחד וטופס בדף אחר
5	חוויית משתמש	רציפה ומהירה יותר, כי אין מעבר מלא בין עמודים	מסודרת וברורה יותר כאשר יש הרבה אזורים או הרבה תוכן
6	ארגון הקוד	כל ה־HTML, CSS וה־JavaScript נמצאים בדרך כלל באותו קובץ או באותו מבנה עמוד	אפשר לחלק קוד ותוכן לפי עמודים, ולכן קל יותר להפריד נושאים גדולים
7	SEO / GEO	צריך להקפיד במיוחד על כותרות, טקסטים סמנטיים ותוכן ברור בעמוד אחד	לרוב קל יותר להסביר למנועי חיפוש ולכלי AI, כי לכל עמוד יש נושא, כותרת וכתובת משלו
8	יתרון מרכזי	מתאים מאוד לפעילות קצרה, משחק, שאלון או כלי תרגול ממוקד	מתאים יותר למערכת גדולה עם הרבה מסכים, נושאים או תכנים
9	חיסרון מרכזי	אם יש הרבה תוכן, העמוד עלול להיות עמוס ומורכב לניהול	המעברים בין דפים פחות רציפים, ולעיתים מרגישים איטיים יותר